Bases de datos 2

Módulo 3

Tabla de contenido

Introducción **3**
Configuración del Entorno de Programación **4**
- **Visual Studio Code (VS Code)** **4**
- **Spring Boot** **8**
  - Generar un Proyecto con Spring Initializr 8
- **Postman** **12**

API REST y Diseño Arquitectónico **14**
- **API REST** **14**
- **Diseño Arquitectónico** **15**
- **Desarrollar el Backend de una API REST** **16**
- **Actividad Asíncrona** **45**

Referencias **46**

Introducción

El desarrollo de un backend eficiente requiere de buenas prácticas de desarrollo de software y herramientas que aseguren la calidad del producto. El uso de un framework robusto como Spring Boot simplifica y acelera el desarrollo de software, reduciendo la complejidad y las configuraciones manuales. Además, su combinación con una arquitectura de software como el Modelo Vista Controlador y Capas permite construir aplicaciones sólidas, escalables y alineadas con las tendencias actuales del desarrollo empresarial actual. Por otro lado, el uso de una base de datos como MongoDB, implementada a nivel local o en la nube, optimiza el funcionamiento de un backend para manejar los datos de manera flexible y eficiente, con una estructura fácil de mantener y que se adapta a las necesidades específicas de la aplicación y del negocio. La unión de todos estos componentes constituye una base sólida para desarrollar aplicaciones robustas, las cuales pueden ser testeadas para verificar el cumplimiento de los requisitos y reducir los riesgos en un entorno de producción.

Configuración del Entorno de Programación

En el contexto del desarrollo de software configurar el entorno de programación está orientado a preparar y establecer todo el conjunto de herramientas y tecnologías necesarias para dar comienzo al proceso de desarrollo. Es decir, tener un ambiente de desarrollo ajustado y operativo que ayude a minimizar los riesgos técnicos que puedan afectar el proceso de desarrollo. Algunas de las herramientas y tecnologías claves para el desarrollo de un backend de una API[1] REST[2], basado en el lenguaje de programación Java haciendo uso del framework Spring Boot y la potencia de una base de datos como MongoDB son:

Visual Studio Code (VS Code)

Es un editor de código abierto desarrollado por Microsoft que puede ser instalado y ejecutado en el escritorio de sistemas operativos como Windows, MacOS y Linux [1], [2]. En VS Code, es posible desarrollar una amplia variedad de aplicaciones de software utilizando diferentes lenguajes de programación, herramientas y depuradores adicionales que facilitan el flujo de trabajo en el ciclo de desarrollo de software. Esto se logra gracias a las diferentes extensiones desarrolladas por Microsoft, empresas de software y comunidades de código abierto, y que están disponibles en el Marketplace[3] de VS Code.

En este sentido, se deberá instalar y configurar VS Code para el desarrollo en Java[4]:

Paso 1: Abrir la ventana del Marketplace en VS Code.

En la ventana principal de VS Code dar clic en el botón .

[1] API: Application Programming Interface
[2] REST: Representational State Transfer
[3] https://code.visualstudio.com/docs/editor/extension-marketplace
[4] https://code.visualstudio.com/docs/java/java-tutorial

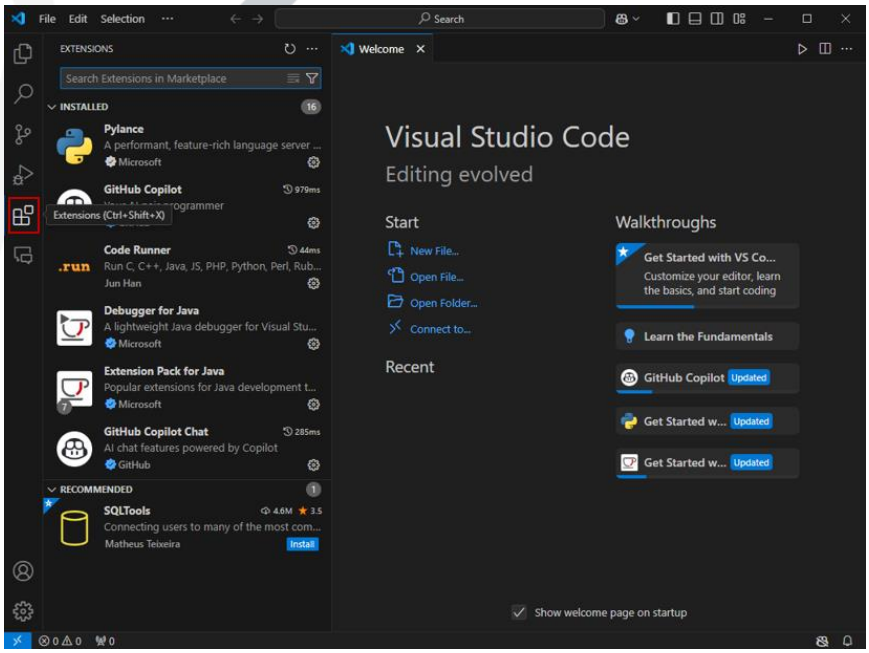

Figura 1. Marketplace de Extensiones

Paso 2: Instalar Extension Pack for Java.
Ir a la ventana de la Marketplace y buscar la extensión escribiendo el nombre completo "Extension Pack for Java" o la palabra "java", como se presenta en la Figura 2. Seleccionar la extensión desarrollada por Microsoft y dar clic en instalar.

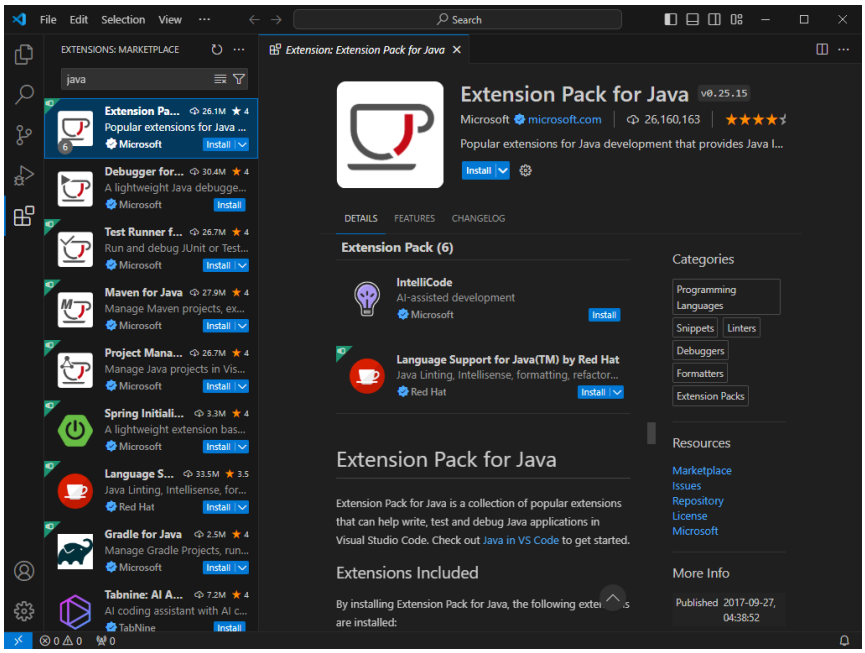

Figura 2. Extension Pack for Java

- **Paso 3: Instalar Java Development Kit (JDK).**
  Si es la primera vez que se programa en Java en el equipo, VS Code sugerirá instalar una JDK por defecto. Por tanto, se deberá dar clic en instalar JDK (Ver Figura 3).

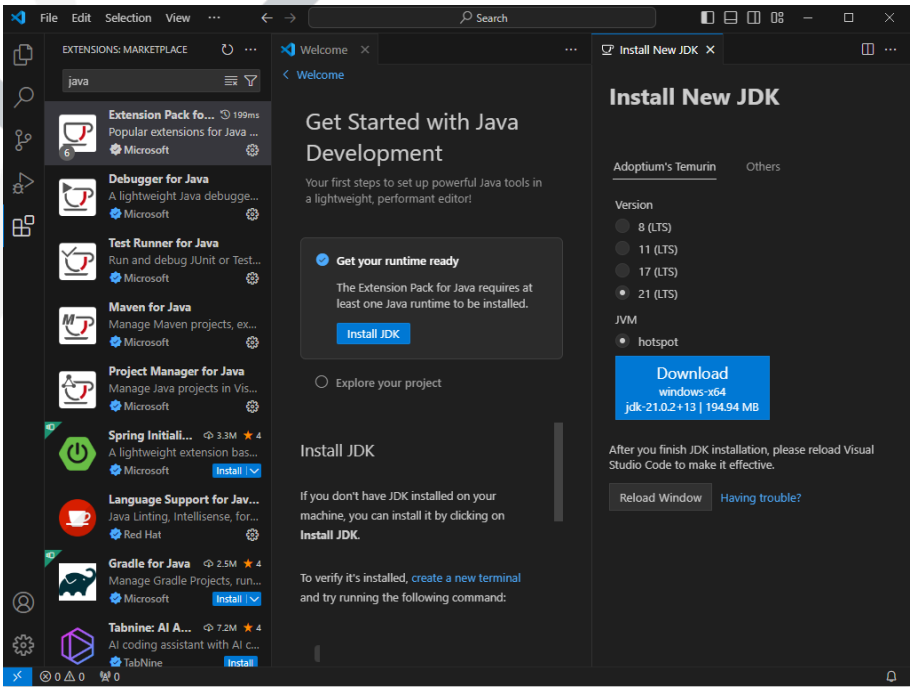

Figura 3. Instalar Nueva JDK

En caso contrario, VS Code detectará de forma automática la JDK que este instalada en el sistema. Para verificar la versión de JDK que está instalada en el equipo, se podrá realizar los siguientes pasos:

A. Ir a la barra de menú de VS Code.
B. Dar clic en Terminal y selecciona New Terminal.
C. En la terminal, escribir el comando “java -version” (Ver Figura 4).

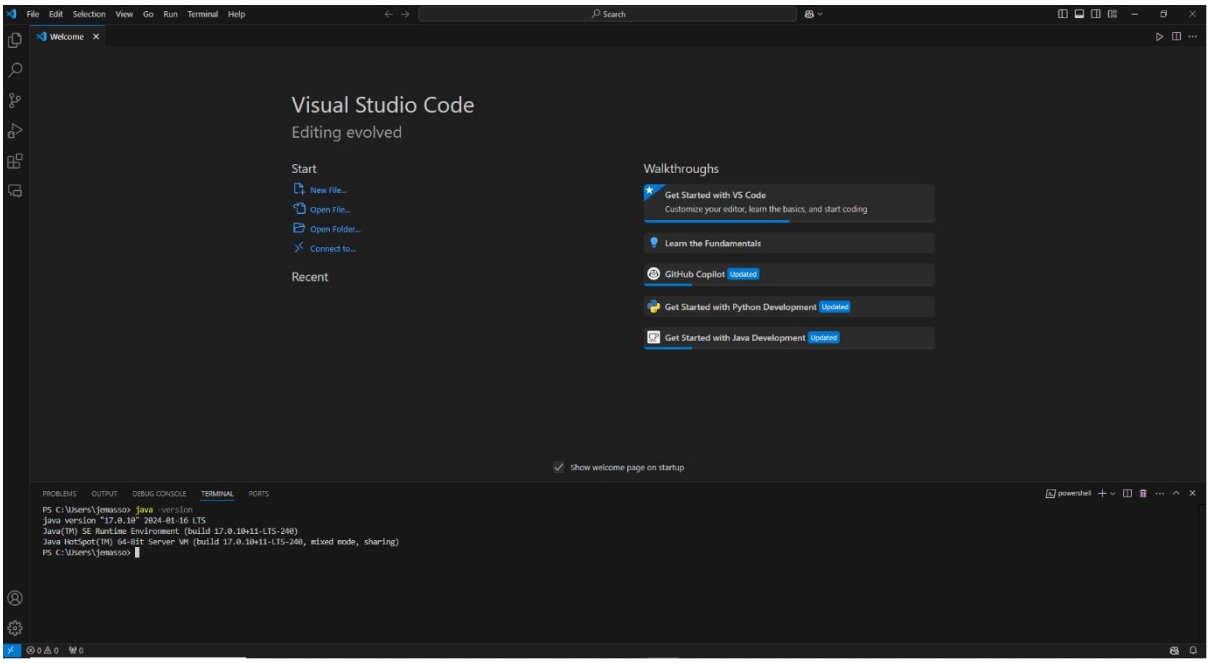

Figura 4. Verificar JDK detectada por VS Code

- **Paso 4: Instalar Spring Initializr Java Support**
  Seleccionar de la lista de extensiones generada en el Paso 2 la desarrollada por Microsoft y dar clic en instalar. Si no aparece en la lista, realiza la búsqueda en la Marketplace de extensiones mediante el nombre completo “Spring Initializr Java Support” o escribiendo “spring”.

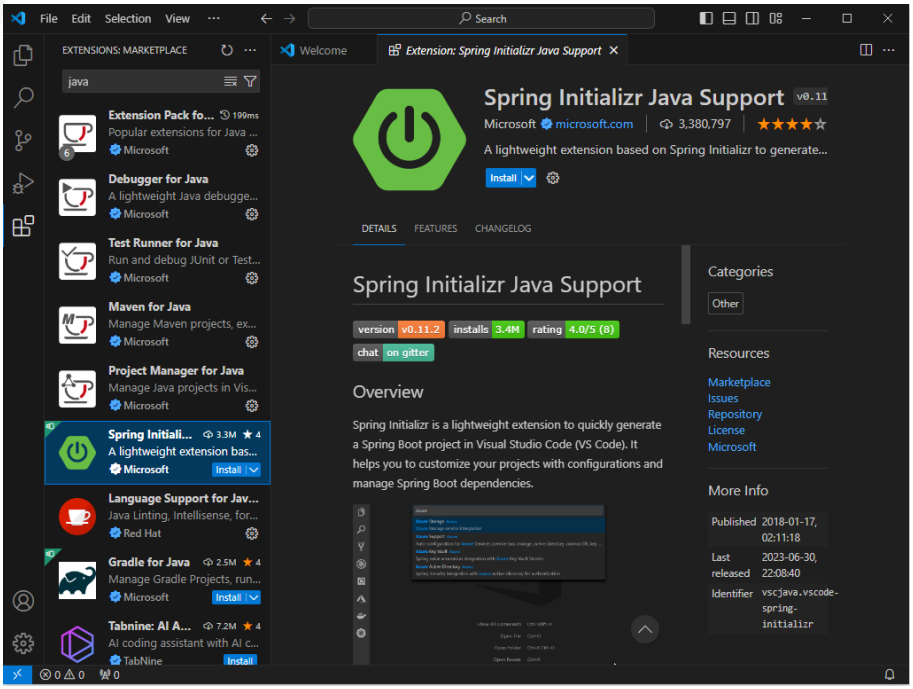

Figura 5. Spring Initializr Java Support

- **Paso 5: Instalar Code Runner.**
  La extensión Code Runner es opcional, pero el tenerla permitirá la ejecución del código de la aplicación de forma rápida. Por tanto, para su instalación se deberá buscarla en la Marketplace mediante el nombre completo “Code Runner” o escribiendo “code”. Seleccionar la desarrollada por Jun Han y dar clic en instalar (Ver Figura 6).

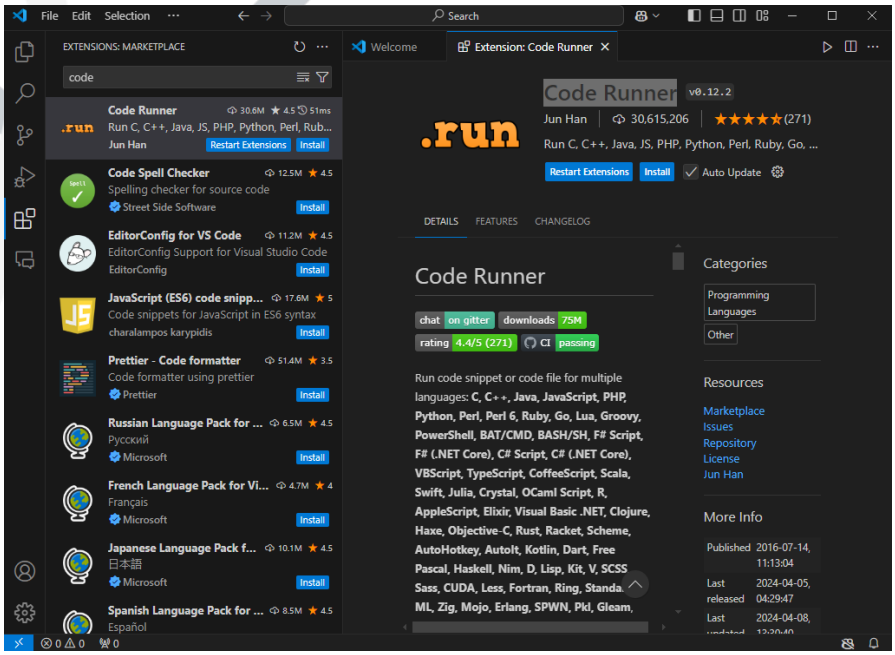

Figura 6. Code Runner

Spring Boot

Spring Boot es uno de los frameworks más populares para el desarrollo de aplicaciones modernas mediante el lenguaje de programación Java. Este framework, ofrece a los desarrolladores un sin número de herramientas que simplifican y aceleran el desarrollo de software al reducir de manera significa la configuración y escritura de código repetitivo, lo que permite a los desarrolladores centrarse en lógica del negocio.

Spring Boot es ideal para la creación aplicaciones modernas como APIs REST, microservicios, aplicaciones sin servidor, sistemas basados en eventos, etc. Este framework incluye un servidor embebido como Tomcat, el cual facilita la ejecución y despliegue de aplicaciones web a través de solicitudes HTTP mediante el puerto 8080. Adicionalmente, permite la gestión eficiente de dependencias para soportar el desarrollo de las aplicaciones y utilizar el patrón de diseño Modelo-Vista-Controlador (MVC) para simplificar su configuración y despliegue.

Generar un Proyecto con Spring Initializr

- **Paso 1: Abrir el sitio web Spring Initializr[5] para crear un proyecto Spring Boot.**
  Se utilizará este sitio ya que nos ofrece una plantilla base que permite realizar una configuración personalizada del proyecto, la cual permite cumplir con las necesidades de la creación de una aplicación. En este caso será una API REST.

[5] https://start.spring.io/

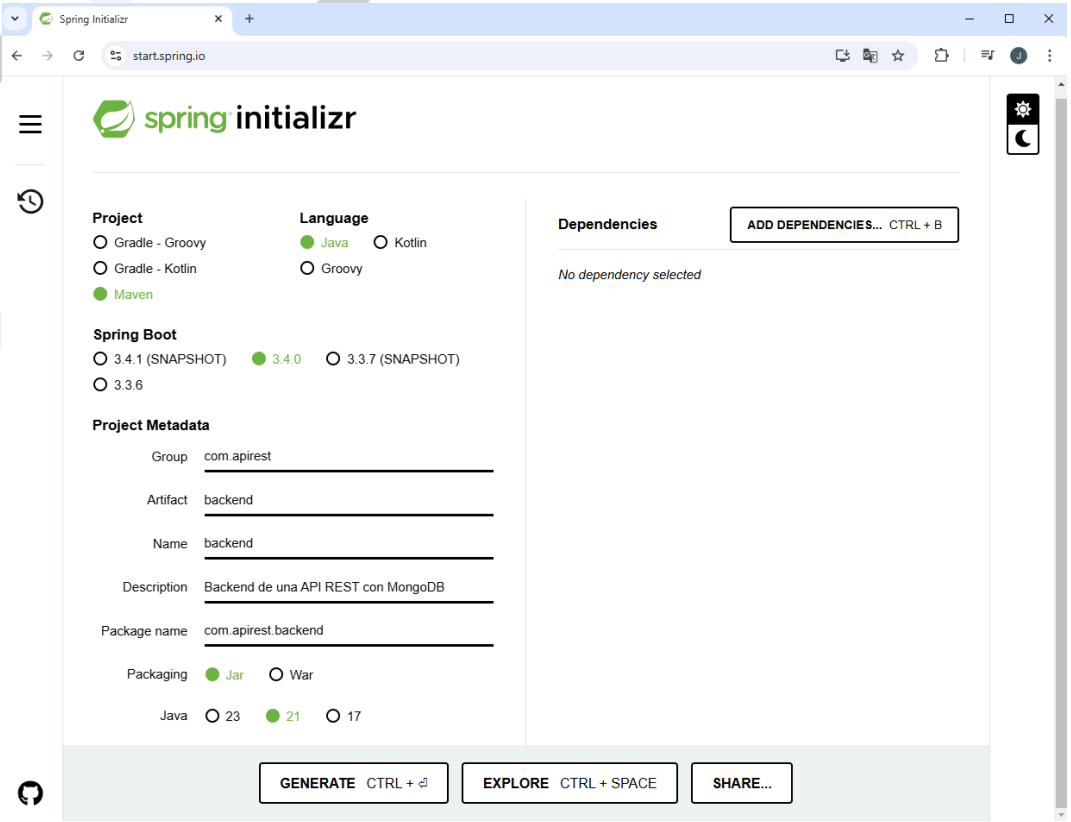

Figura 7. Plantilla base de Spring Initializr

- **Paso 2: Configurar la gestión de las dependencias y la estructura de construcción del proyecto.**
  Seleccionar o dejar por defecto la herramienta Maven. Esta herramienta permite gestionar de forma automática la construcción del proyecto. Garantizando que el proceso de construcción sea repetible y consistente. Una vez se cree el proyecto se generará el archivo “pom.xml” (Project Object Model), el cuál contendrá todas las dependencias y configuraciones que se hayan realizado al proyecto.

- **Paso 3: Diligenciar la metadata del proyecto.**
  Especificar el grupo o dominio bajo el cual se agruparán todos los componentes de la aplicación. Establecer el identificador del proyecto a través del “Artifact”. Este elemento servirá para nombrar el archivo de empaquetamiento que se generará al compilar la aplicación. Brindar el nombre del proyecto, usualmente es el mismo del “Artifact”. Brindar una descripción breve del proyecto que permitirá identificar qué tipo de desarrollo se desea realizar. Finalmente, especificar el nombre del paquete raíz que será donde se coloque el código del proyecto.

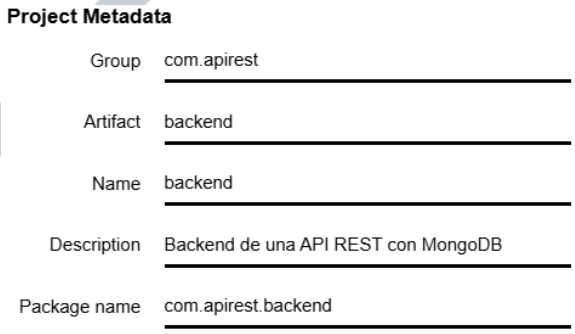

Figura 8. Metadata del Proyecto

- **Paso 4. Especificar el empaquetamiento y la JDK.**
  Este paso es crucial ya que se especificará como se construirá y ejecutará el proyecto. En este caso se utilizará el "jar", ya que se creará una aplicación autónoma que utilizará el servidor embebido de spring en este caso Tomcat. Seleccionar la JDK según el entorno de desarrollo (equipo de desarrollo) y/o equipo de ejecución de la aplicación (servidor o máquina de producción).

- **Paso 5: Agregar dependencias.**
  Seleccionar las dependencias o componentes necesarios para la creación del backend de una API REST. En este caso se emplearán las siguientes dependencias:

  A. **Spring Boot DevTools**: Esta dependencia proporciona herramientas y funcionalidades que mejoran la productividad y eficiencia del desarrollador durante el desarrollo de la aplicación.
  B. **Spring Web**: Proporciona los elementos necesarios para el desarrollo de aplicaciones web, incluyendo soporte para la gestión de solicitudes HTTP y la creación de controladores REST.
  C. **Spring Data MongoDB**: Proporciona una capa de abstracción sobre el controlador de conexión MongoDB Java Driver. Esta capa simplifica las operaciones y facilita el desarrollo de aplicaciones Spring al ofrecer una interfaz coherente y fácil de usar para interactuar con la base de datos MongoDB.
  D. **Lombok**: Este proyecto tiene como finalidad reducir la escritura del código, permitiendo generar de forma automática la creación de métodos comunes en clases Java, como getters, setters y constructores.

  Nota*: Para adicionar las dependencias se deberá dar clic en el botón "ADD DEPENDENCIES… CTRL +B". En la ventana emergente, se podrá colocar e*l nombre completo de la dependencia o un nombre corte. Por ejemplo, para agregar la dependencia Spring Data MongoDB, escribir mongo en el cuadro d búsqueda, seleccionar Spring Data MongoDB y luego dar clic sobre ella para que integre a la configuración. (Ver Figura 9)

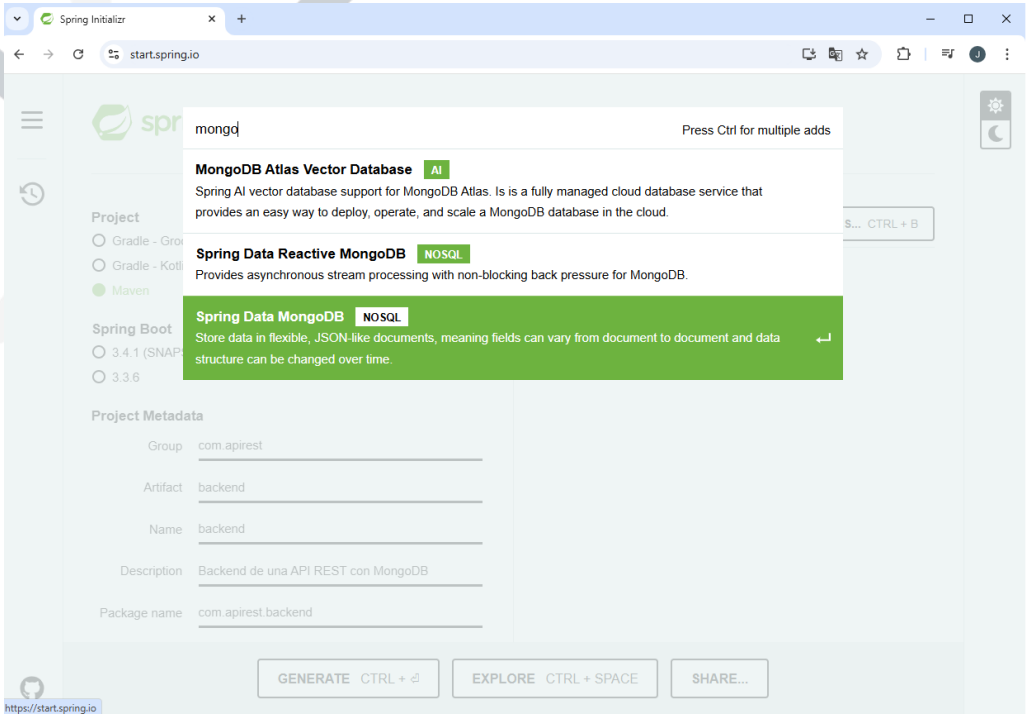

Figura 9. Adicionar Dependencias

- **Paso 5: Generar el proyecto.**
  Una vez se termine de diligenciar la plantilla base del proyecto en spring, se deberá dar clic en el botón “GENERATE CTRL +⏎ ”. El sitio generará un archivo “.zip” con el nombre brindado al proyecto “backend.zip”. Esta carpeta se deberá descomprimir y guardar en el área de trabajo que se emplee para los proyectos personales. En la Figura 10, se puede observar la configuración definitiva realizada para el proyecto que se desarrollará en esta guía.

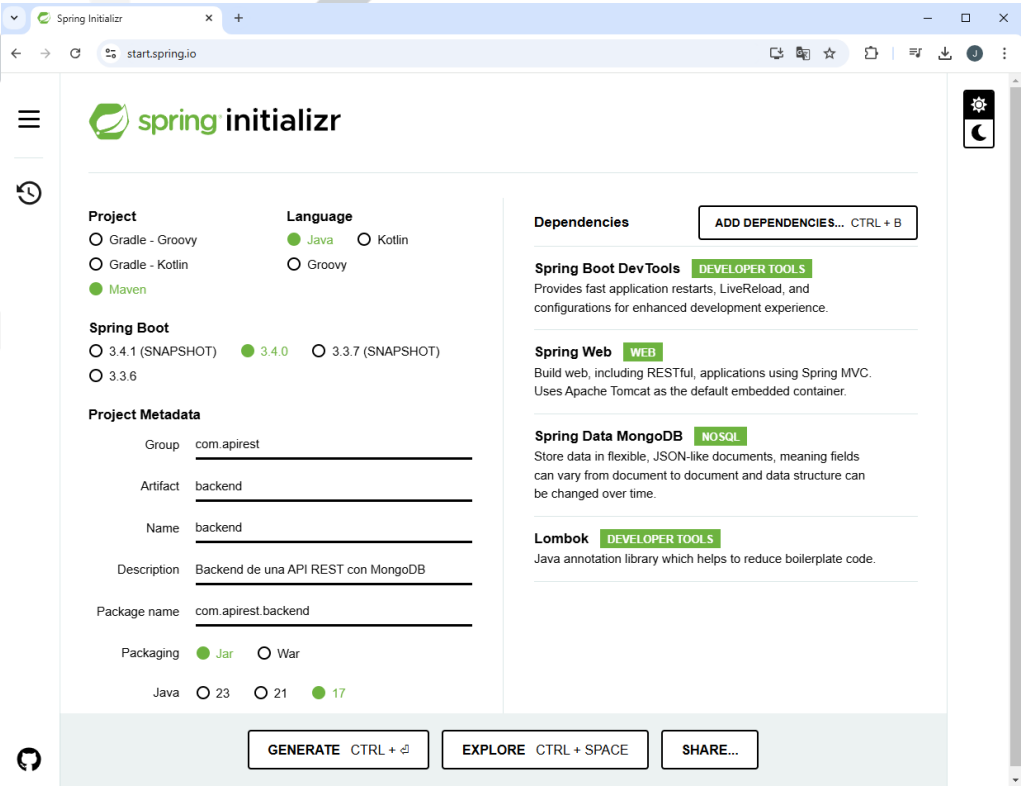

Figura 10. Configuración Final del Proyecto

Postman

Postman es una herramienta empleada por equipos de aseguramiento de calidad (QA, en sus siglas en inglés) y desarrolladores software, con el fin de interactuar, probar y documentar las APIs de manera eficiente y colaborativa.

Paso 1: Descargar e instalar Postman[6].

Ir al sitio oficial de y descargar la APP de Postman según el sistema operativo (Windows, macOS o Linux). Una vez descargado seguir las instrucciones del programa para su instalación.

Paso 2: Crear una cuenta en Postman[7]

Una vez instalado en el equipo se deberá crear una cuenta en Postman. Para hacerlo se pueden seguir dos caminos: abrir la aplicación (Ver Figura 11) o dirigiéndose a la página oficial de Postman (Ver Figura 12) en donde se deberá dar clic en "Sign Up", diligenciar el formulario y seguir el proceso indicado.

[6] https://www.postman.com/downloads/
[7] https://shorturl.at/UaEAL

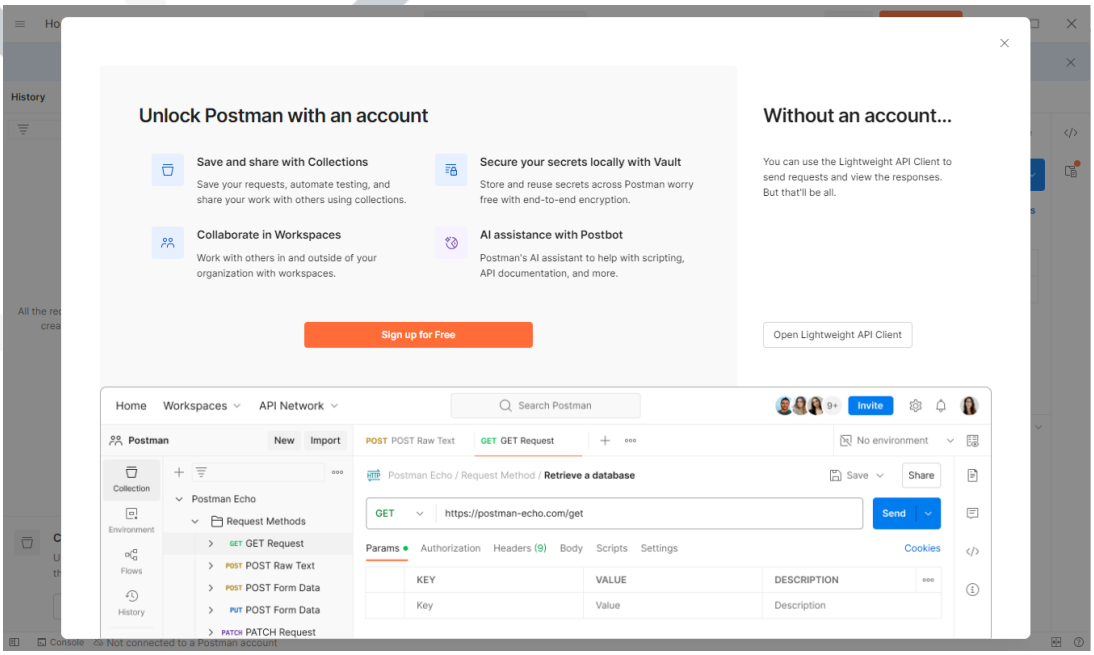

Figura 11. Inicio de Postman en el Equipo

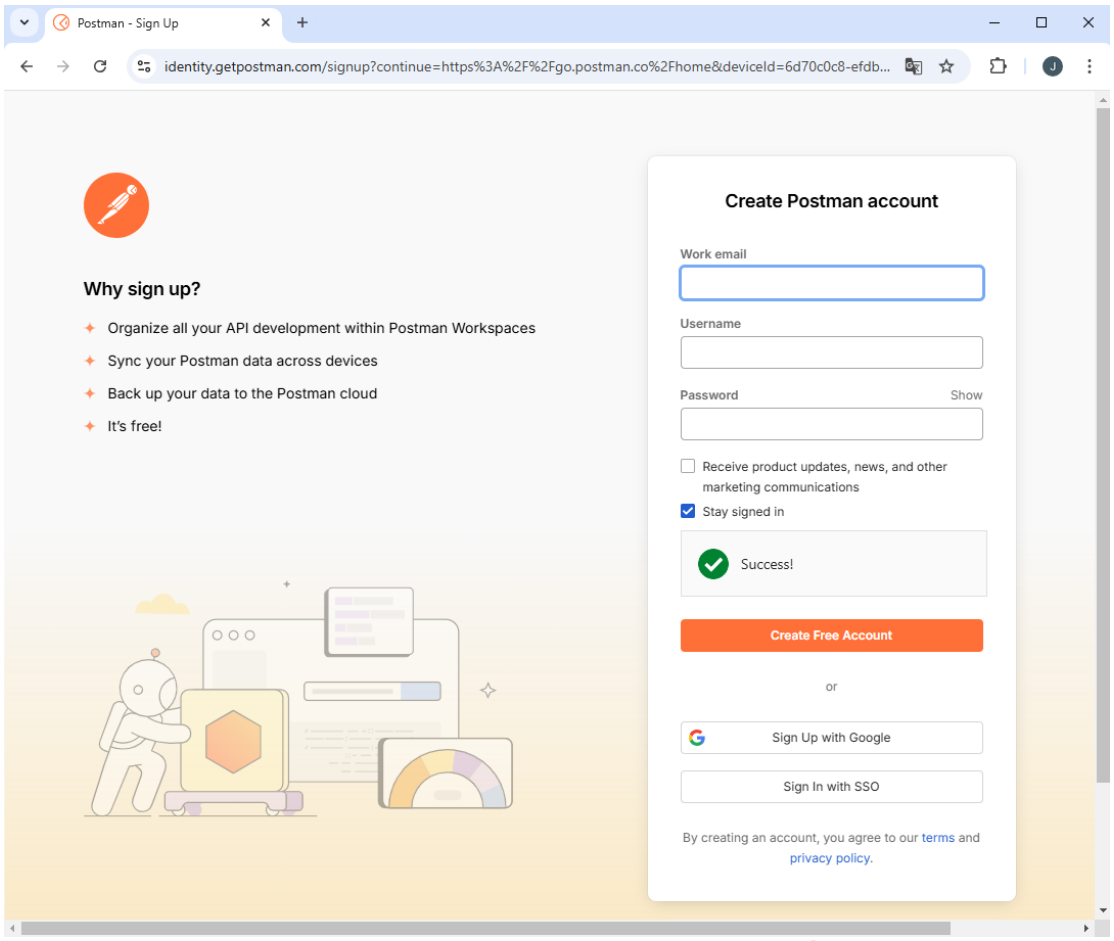

Figura 12. Crear una Cuenta de Postman Vía Web

Paso 3: Iniciar Postman.
Una vez este creada la cuenta, se deberá autorizar a la API para utilizar a Postman en el equipo. De esta manera quedará lista para realizar cualquier tipo de interacción con una aplicación.

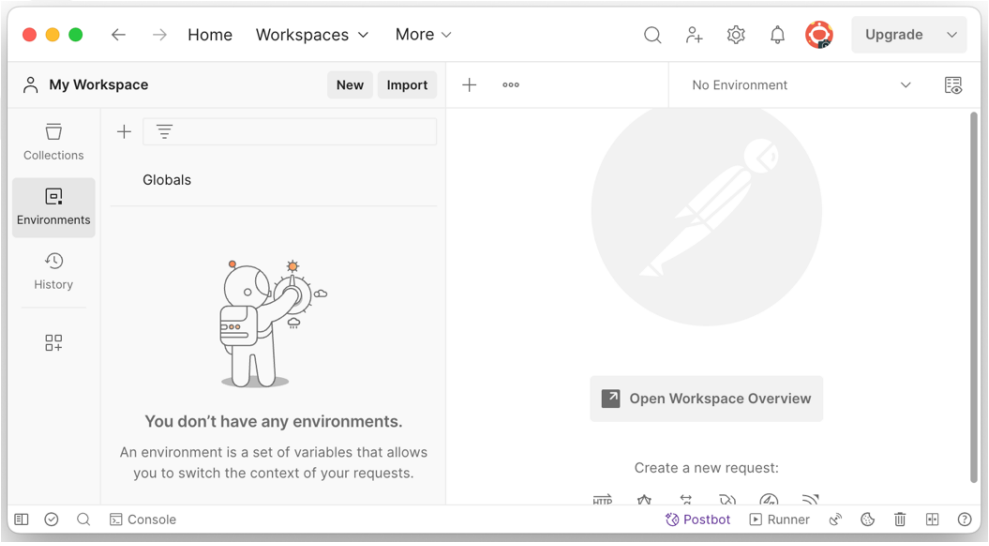

Figura 13: Entorno Principal de Postman

API REST y Diseño Arquitectónico

Ahora es el momento de comprender de manera general los fundamentos básicos sobre lo que es una API REST y la utilidad de los patrones de diseño arquitectónico para la creación de aplicaciones software.

API REST

Una Interfaz de Programación de Aplicaciones (en sus siglas en inglés API) es un conjunto de definiciones, protocolos y herramientas diseñadas para integrar y comunicar diferentes aplicaciones o sistemas. Una API es un mediador que permite conectar a un usuario con los diferentes recursos o servicios informáticos que ofrece una aplicación (por ejemplo, web, móvil, etc.).

Cuando una API adopta un estilo arquitectónico como REST (Representational State Transfer) incorpora principios que garantizan una comunicación segura, confiable y eficiente basada en el protocolo HTTP [3], [4]. REST define cómo debe operar una API, permitiendo el intercambio de información en formatos como JSON o XML. Además, soporta las operaciones CRUD mediante solicitudes HTTP, a través de verbos estándar como son:

- **GET**: Permite la recuperación de datos.
- **POST**: Se utiliza para la creación de datos.
- **PUT**: Facilita la actualización de datos.
- **PUTCH**: Permite la actualización parcial de datos.
- **DELETE**: Permite eliminar datos de forma definitiva.

Diseño Arquitectónico

El diseño arquitectónico a nivel del desarrollo de software hace referencia al uso de patrones arquitectónicos que permiten brindar una estructura general y reutilizable para abordar la implementación de una aplicación. Como se mencionó anteriormente, Spring Boot hace uso del patrón MVC para soportar el desarrollo de aplicaciones web. Este patrón permite gestionar la interacción entre los usuarios y la aplicación, mediante una interfaz gráfica de usuario. En este sentido los principales componentes del MVC son:

- **Modelo (Model)**: Este componente representa la lógica datos y del negocio. Es responsable de procesar las solicitudes y mantener el estado de la aplicación.
- **Vista (View):** Este componente se encarga de proporcionar la interfaz de usuario para interactuar con la aplicación y mostrar la información al usuario.
- **Controlador (Controller)**: Este componente proporciona toda la lógica necesaria para recibir, validar, direccionar las solicitudes HTTP, y gestionar las respuestas hacia la vista. Es decir, es el intermediario entre la vista y el modelo.

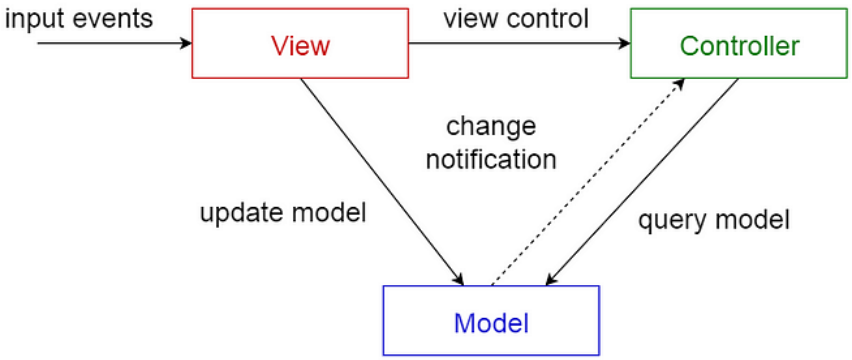

Figura 14. Patrón MVC (Fuente: https://bit.ly/46CPowR)

En el desarrollo de aplicaciones web con Spring Boot, es común integrar los patrones MVC y Capas (Layered) para garantizar una arquitectura sólida, escalable, modular y mantenible. El patrón MVC se encarga de cumplir con las responsabilidades de los componentes externos (interacción entre el usuario y la aplicación), mientras que el patrón Capas se encargará de estructurar y organizar la lógica interna, es decir, la gestión de la lógica del negocio, el acceso a datos, la manipulación y transferencia de datos.

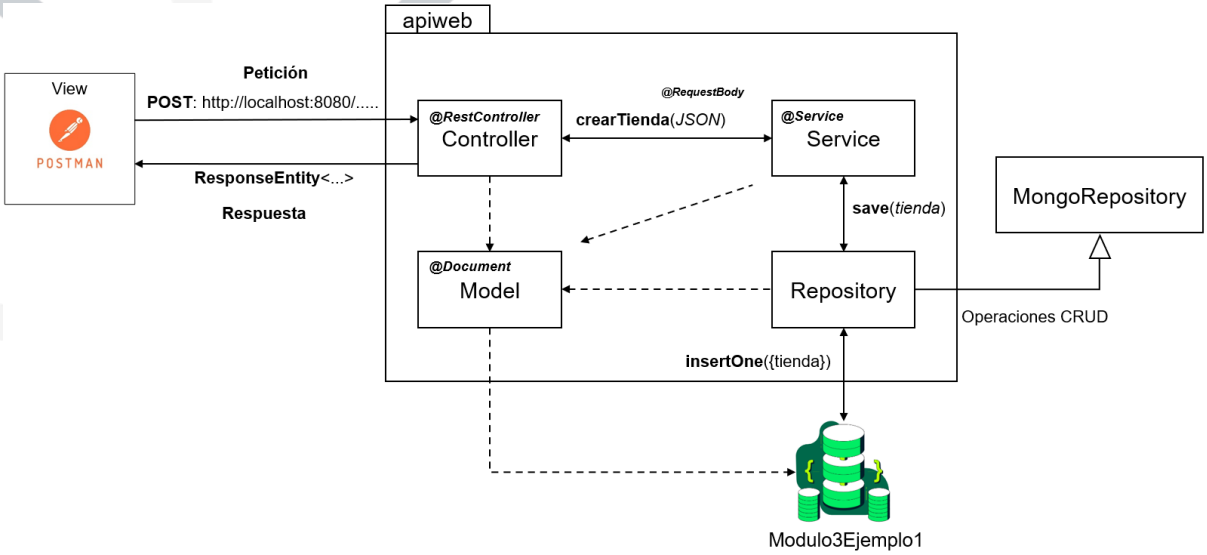

Figura 15. Flujo de Trabajo de la Aplicación

En la Figura 15, se puede observar el flujo de trabajo básico, en donde se pueden apreciar los principales componentes del backend y sus respectivas responsabilidades. A continuación, se describe de forma general el flujo de interacción de los diferentes componentes:

- El usuario realiza una petición HTTP (ej. POST: http://localhost:8080/UAO/apirest/Tiendas/ crearTienda) desde Postman (View).
- El Controller recibe la solicitud, valida los datos y envía a la capa Service.
- El Service ejecuta la lógica de negocio correspondiente y solicita las acciones necesarias a la capa Repository.
- El Repository accede a la base de datos haciendo uso de las funcionalidades a través de MongoRepository y los retorna a la capa Service.
- El Service procesa los datos recibidos y los retorna a la capa Controller.
- Finalmente, el Controller envía una respuesta con los datos al usuario en formato JSON para que sean presentados al usuario en Postman.

Finalmente, la integración de los patrones MVC y Capas, facilita la extensibilidad y el mantenimiento de las aplicaciones, ya que promueve una separación clara de responsabilidades y asegura que cada componente tenga un propósito bien definido para su correcto funcionamiento.

Desarrollar el Backend de una API REST

En este ejercicio práctico, se desarrollará el backend de una API REST utilizando Spring Boot y MongoDB. La arquitectura de software adoptada para estructurar y organizar el código integrará los patrones MVC y Capas, lo que garantizará una solución escalable, modular y mantenible. A continuación, se detallan los pasos a seguir para llevar a cabo el desarrollo del backend y pruebas para validar su correcto funcionamiento.

Paso 1: Crear la base de datos en MongoDB Compass

Modelar en Hackolade la base de datos que se presenta en la Figura 16. Se deberá garantizar que las colecciones tengan como nivel de validación el moderado, se controlen el manejo de error y permitan la adición futura de propiedades.

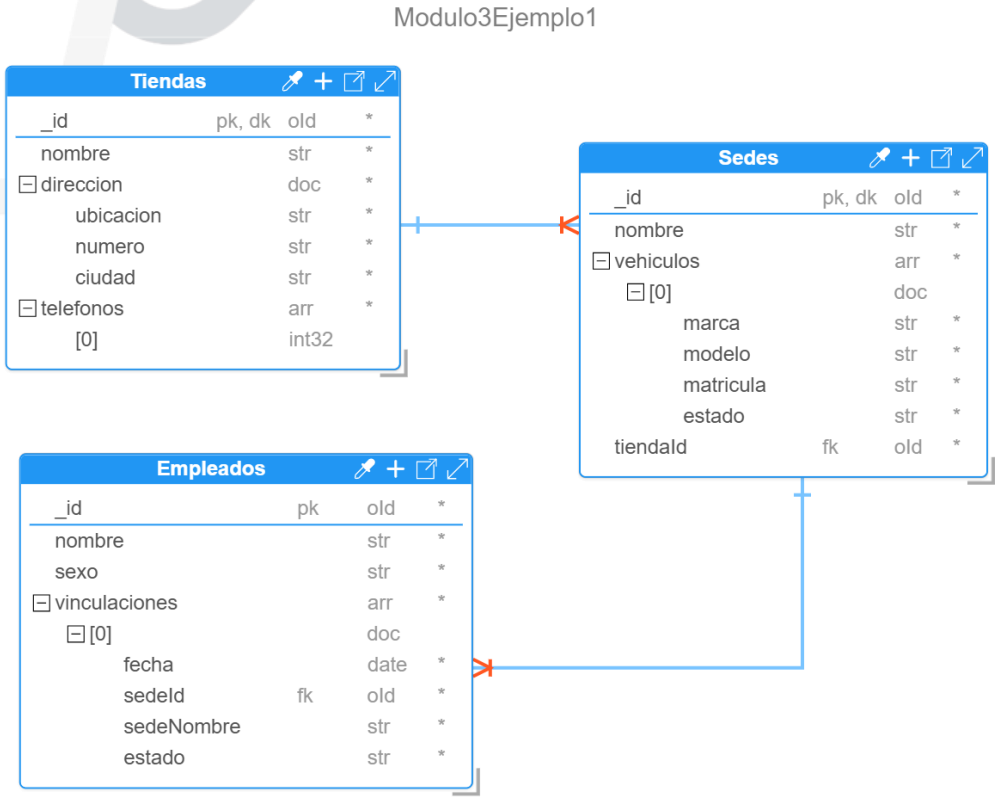

Figura 16. Base de Datos Modulo3Ejemplo1

Nota: El estado para los vehículos de la sede son activo, inactivo, baja

Paso 2: Crear el proyecto de Spring Boot

Para crear el proyecto, utilizaremos la herramienta web "Spring Initializr" siguiendo la estructura definida en la Figura 10.

Paso 3: Abrir el proyecto de Spring en VS Code.

Abrir VS Code y dar clic en el icono

de la ventana de bienvenida. Ir a la barra de menú y seleccionar "File" y luego "*Open Folder…*". Paso siguiente, buscar la carpeta que contiene el proyecto generado en la web Spring Initializr, esta deberá estar descomprimida. Una vez abierto el proyecto aparecerá una ventana emergente y se deberá especificar que se confía en la procedencia del proyecto y se deberá dar clic en el en el botón "Yes, I trust the authors" (Ver Figura 17).

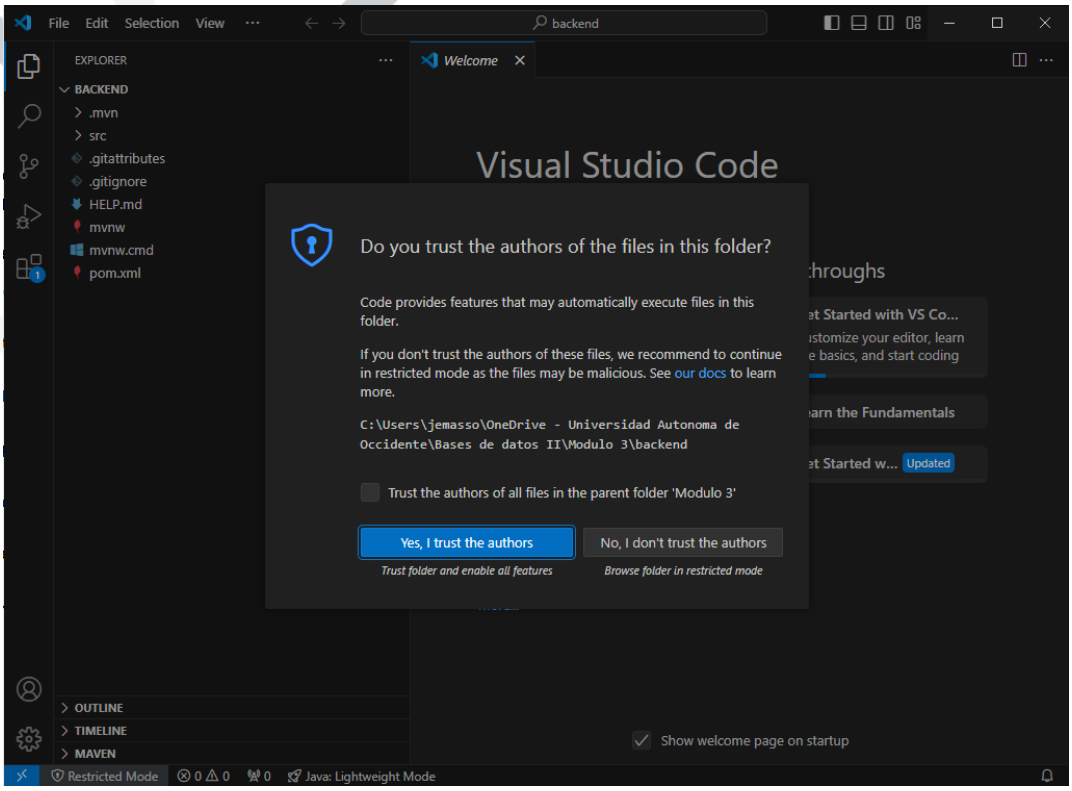

Figura 17. Abrir el Proyecto Backend en VS Code

Paso 4: Crear las capas del proyecto backend con sus respectivos recursos

Con el fin de seguir la estructura de la arquitectura software a través de la integración de los patrones MVC y Capas, se creará cada una de las carpetas que representarán las capas del proyecto backend, cada una con responsabilidades específicas. Las capas que se implementarán son:

- **Modelo (Model)**: Contendrá las clases que mapearan las colecciones de nuestra base de datos en MongoDB.
- **Respositorio (Repository)**: Permitirá gestionar las interacciones con la base de datos. Proveerá las funcionalidades necesarias para llevar a cabo las operaciones CRUD y garantizará la persistencia de los datos mediante el uso de interfaces y consultas específicas mediante la creación de un pipeline de agregaciones.
- **Servicio (Service)**: Encapsulará toda la lógica del negocio, a través de la declaración e implementación de las funcionalidades claves de la aplicación. Interactuará con la capa Repositorio para cumplir con su funcionamiento.
- **Controlador (Controller)**: Se encargará de dar manejo a las solicitudes HTTP entrantes desde la capa Vista. Además, interactuarán con la capa de Servicio para aplicar la lógica de negocio, y generarán las respuestas adecuadas.
- **DTO**: Facilitará la transferencia de datos entre capas. Su propósito principal en el backend es soportar la creación de consultas personalizadas y estructurar datos que no puedan ser gestionados por la capa Modelo.
- **Vista (View)**: Debido a que solo se desarrollará el backend de la API REST, se utilizará la herramienta Postman para soportar esta capa. Postman permitirá realizar la interacción con el backend mediante solicitudes HTTP y servirá como medio para visualización de los resultados.

Paso 4.1: Crear la capa "Model"

Para su creación, se deberá dar clic derecho en la ruta del proyecto "java/com/apirest/backend", seleccionar la opción "*New Folder…*" (Ver Figura 18), brindar el nombre Model y presionar enter para que se cree.

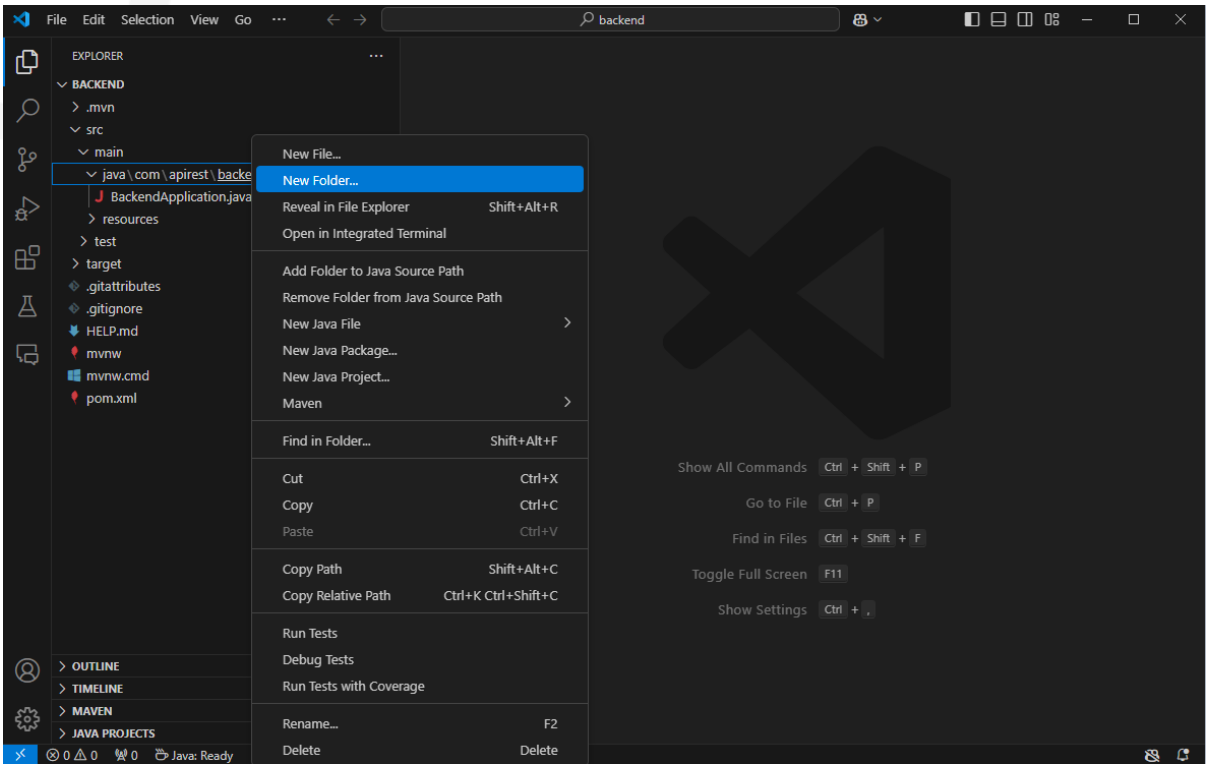

Figura 18. Crear una Carpeta en VS Code

Una vez creada la carpeta Model, se podrá crear la Clase "TiendasModel.java" la cual mapeará la colección Tiendas de la base de datos. Para crear esta clase se deberá dar clic derecho en la carpeta Model y seleccionar la opción "*New File…*" (Ver Figura 19), brindar el nombre "TiendasModel.java" y presionar enter para que se cree.

Nota*: Es importante brindar la extensión ".java" para garantizar que se creará un archivo* de Java. Sino se creará un archivo de texto por defecto.

Una vez creada la clase "TiendasModel", se deberán definir los atributos que representarán los datos de la colección en MongoDB, estos son: _id, nombre, dirección (con los campos ubicación, número y ciudad) y teléfonos.

Descripción general de los atributos:

- id: Identificador principal de la colección, marcado con la anotación @Id.
- dirección: Documento embebido. Para su manejo se debe crear una clase independiente nombrada como "DireccionTienda" que contendrá los atributos: ubicación, número y ciudad. En la clase "TiendasModel", se deberá crear entonces un objeto dirección el atributo de tipo "DirecciónTienda" para cumplir con el documento embebido.

- teléfonos: Es un atributo multivaluado. Para su se representación, se empleará una lista (List<Integer>) que permitirá almacenar múltiples números de teléfono para una tienda.

La implementación final de la clase “TiendasModel” se puede observar en la Figura 21, mientras que el diseño de la clase Dirección se presenta en la Figura 20.

Descripción de las anotaciones utilizadas en las clases:

Anotaciones de Spring Boot

- **@Document:** Permite especificar que la clase mapeará una colección de una base de datos en MongoDB.
- **@Id**: Esta anotación nos permite marcar que un atributo se comportará como identificador principal.

Anotaciones de Lombok

Lombok simplifica la creación de clases eliminando la necesidad de escribir código repetitivo (getters, setters, constructores, etc). En este caso, se utilizan:

- **@Data**: Permite que se genere de forma automática los getters y setters de una clase, entre otros.
- **@AllArgsConstructor**: Crear un constructor con todos los atributos de la clase como parámetros de entrada.
- **@NoArgsConstructor**: Crear un constructor por defecto, es decir sin parámetros.

Estas anotaciones son esenciales para incluir funcionalidades proporcionadas por las dependencias del proyecto y el framework Spring Boot, simplificando la implementación y mejorando la legibilidad del código.

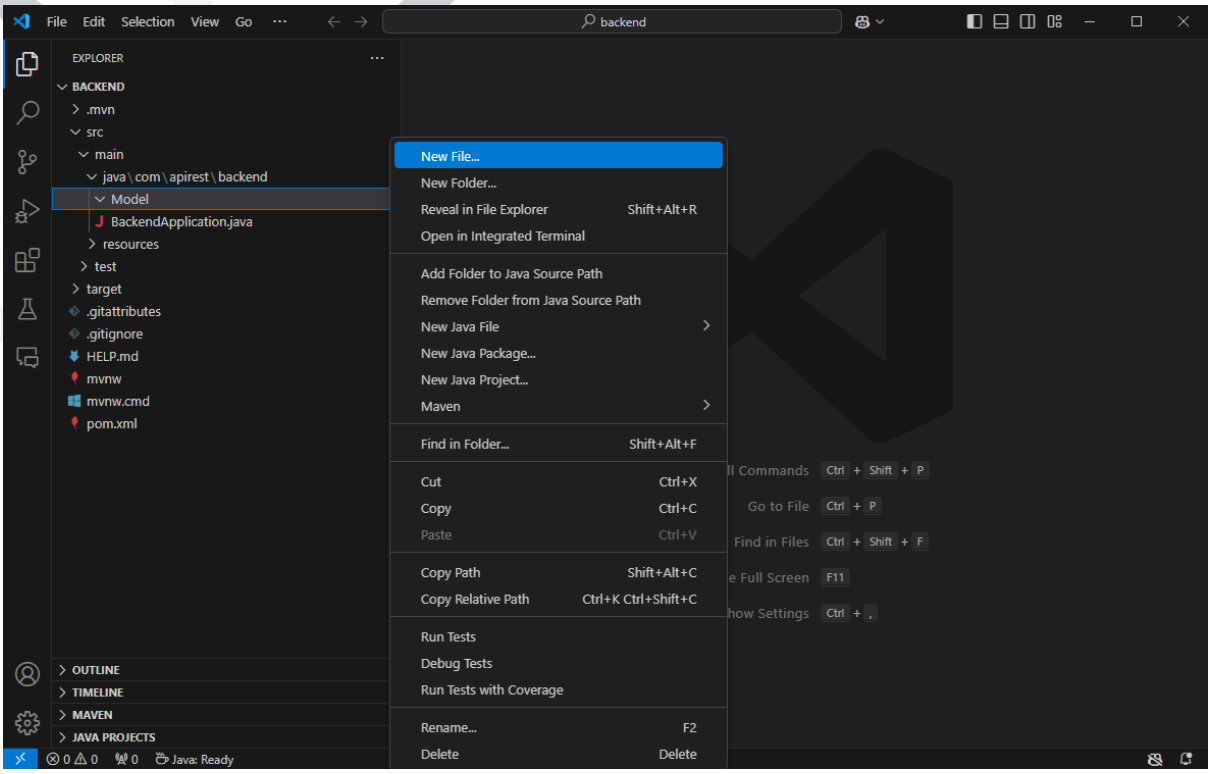

Figura 19. Crear una Archivo Java en VS Code

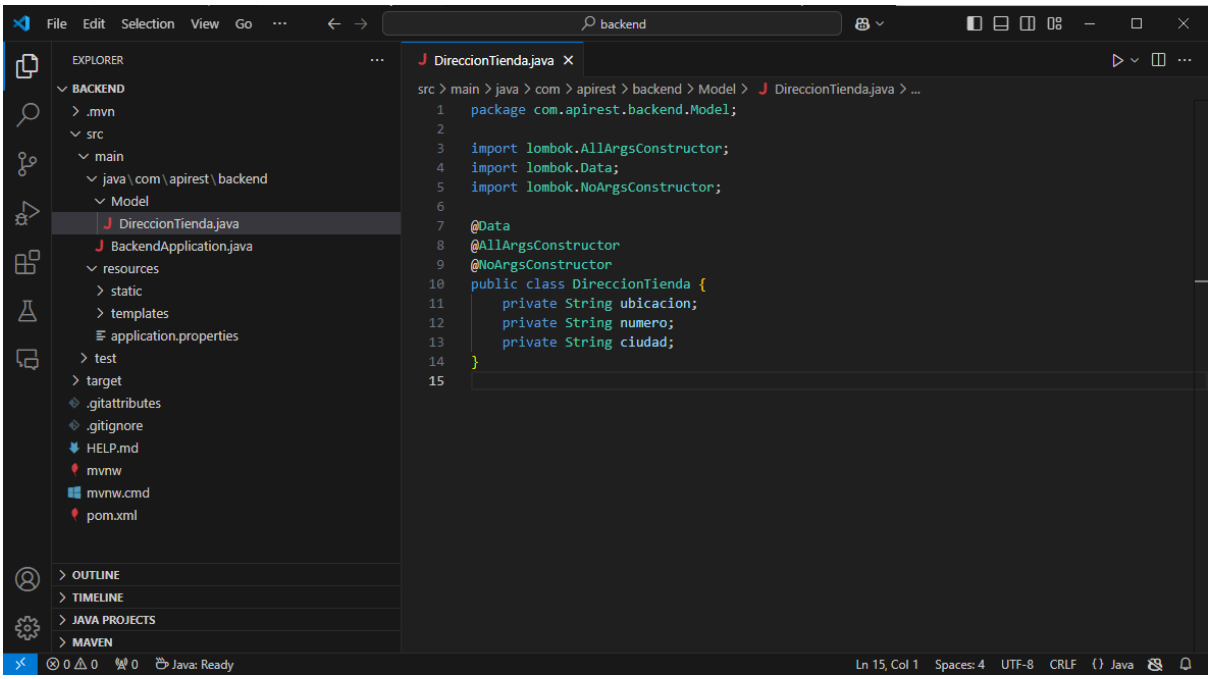

Figura 20. Clase DireccionTienda

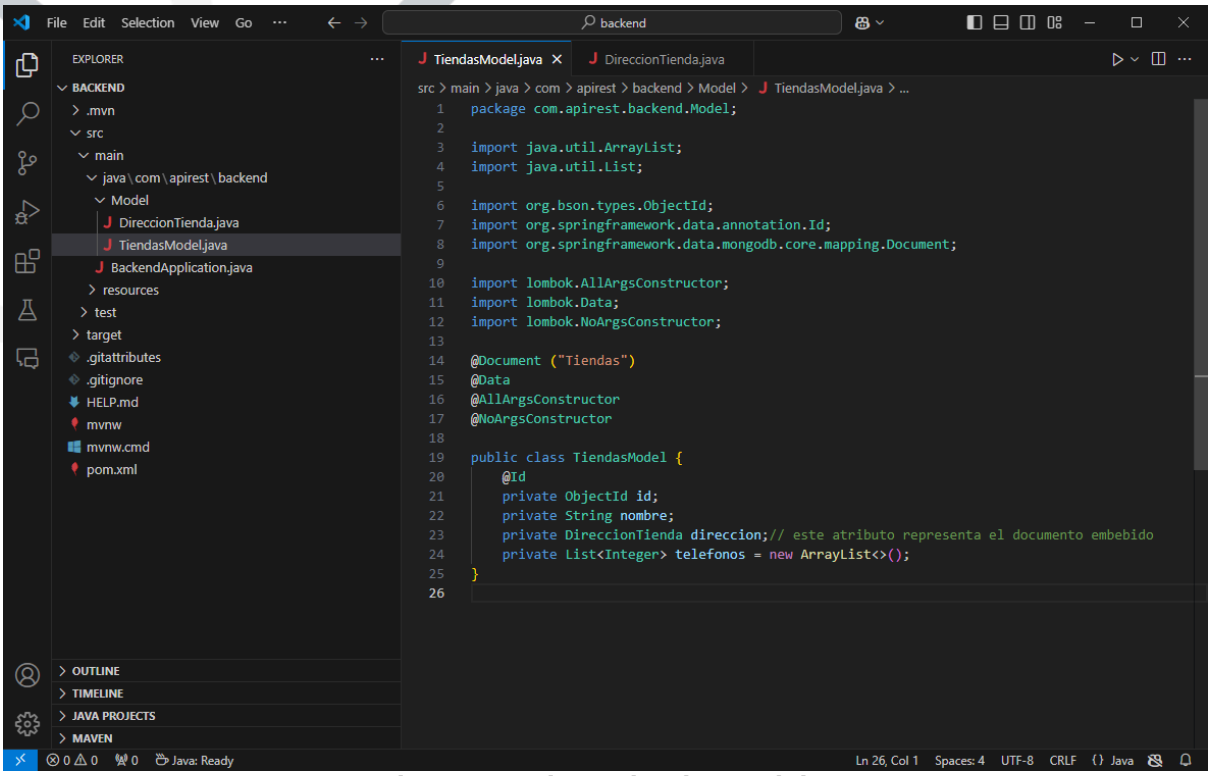

Figura 21. Clase TiendasModel

Paso 4.2: Crear la capa "Repository"

Crear la carpeta "Repository" en la ruta del proyecto "java/com/apirest/backend", y dentro de esta crear la interfaz "ITiendasRepository.java", que permitirá gestionar las interacciones con la base de datos. Es importante que al crear el nuevo archivo especifiquemos que el tipo de estereotipo que representará es una "interface" (Ver Figura 22).

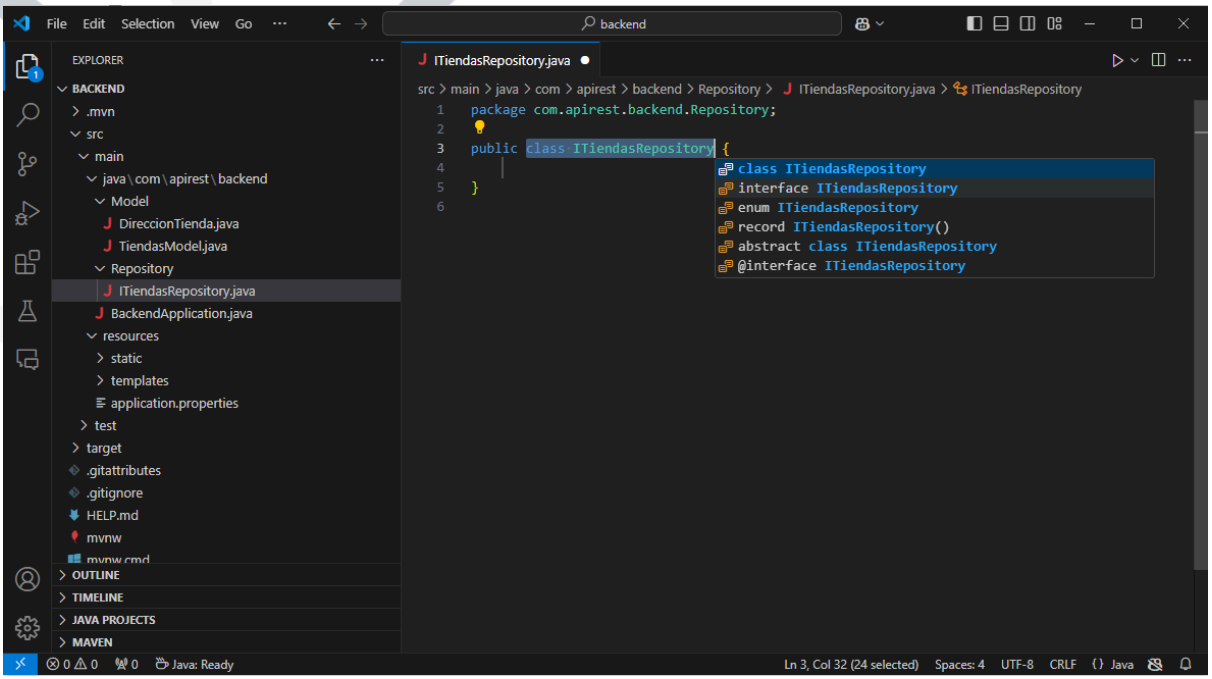

Figura 22. Crear una Interface

La interfaz “ITiendasRepository” deberá extender de “MongoRepository”, para heredar todas las operaciones que se pueden hacer sobre la base de datos y que están predefinidas en Spring Boot. Por tanto, se deberá especificar los siguientes componentes en el “MongoRepository” <“Nombre de la clase que mapea la colección”, “nombre de la clase que representa el tipo de dato del identificador principal de la colección”> (Ver Figura 23).

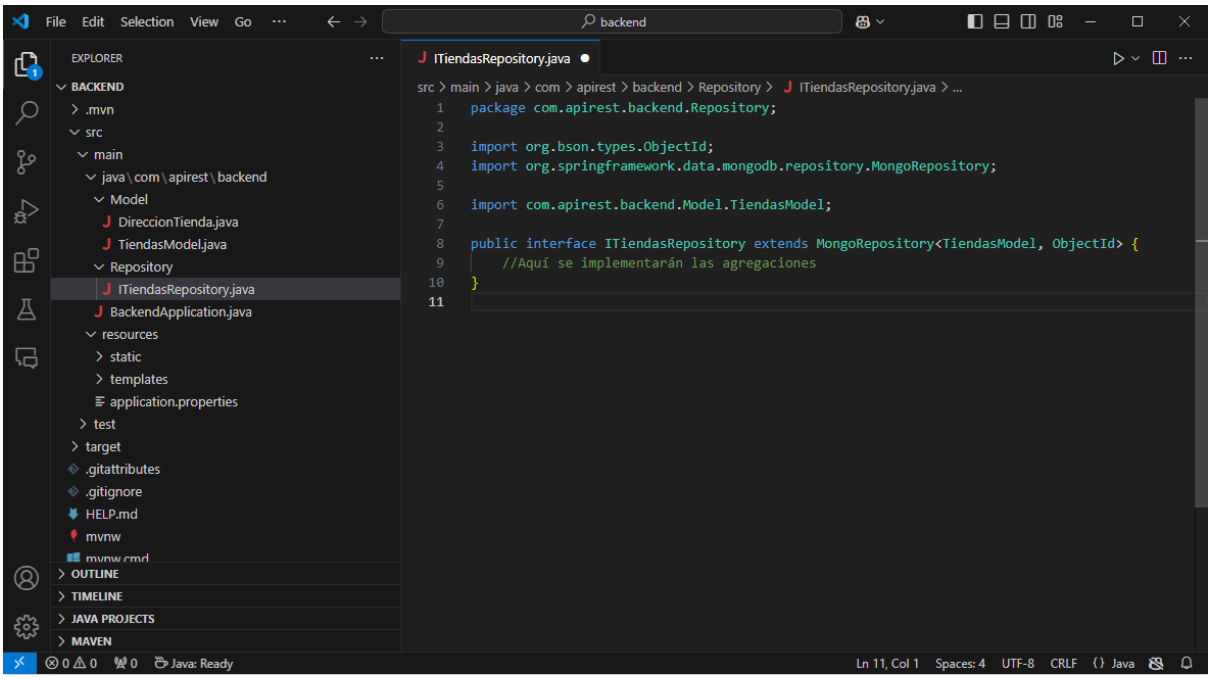

Figura 23. Interfaz ITiendasRepository

Paso 4.3: Crear la capa “Service”

En los proyectos de Spring Boot, una buena práctica de programación es desacoplar las responsabilidades para mejorar la escalabilidad, mantenibilidad de código y hacer el código más limpio, etc. Para cumplir con esta buena práctica de desarrollo se creará la carpeta “Service” en la ruta del proyecto “java/com/apirest/backend”, y dentro de esta los siguientes componentes:

- La interfaz “ITiendasService”, definirá el contrato de las funcionalidades que ofrecerá el servicio relacionado con la clase “TiendasModel”. Uno de sus principales propósitos es establecer el puente comunicación con el controlador a través del uso de inyecciones de dependencias.
- La clase “TiendasServiceImp”, se encargada de implementar todas las funcionalidades definidas en la interfaz “ITiendasService”. Para identificar en el proyecto que esta clase representa un componente de servicio, se debe utilizar la anotación “@Service”. Además, se utilizará la anotación “@Autowired” para realizar la inyección de dependencias con “*ITiendasRepository”*, lo que permitirá acceder a las operaciones CRUD y otras interacciones necesarias con la base de datos.

La implementación de estos componentes se detalla en las Figuras 24 y 25.

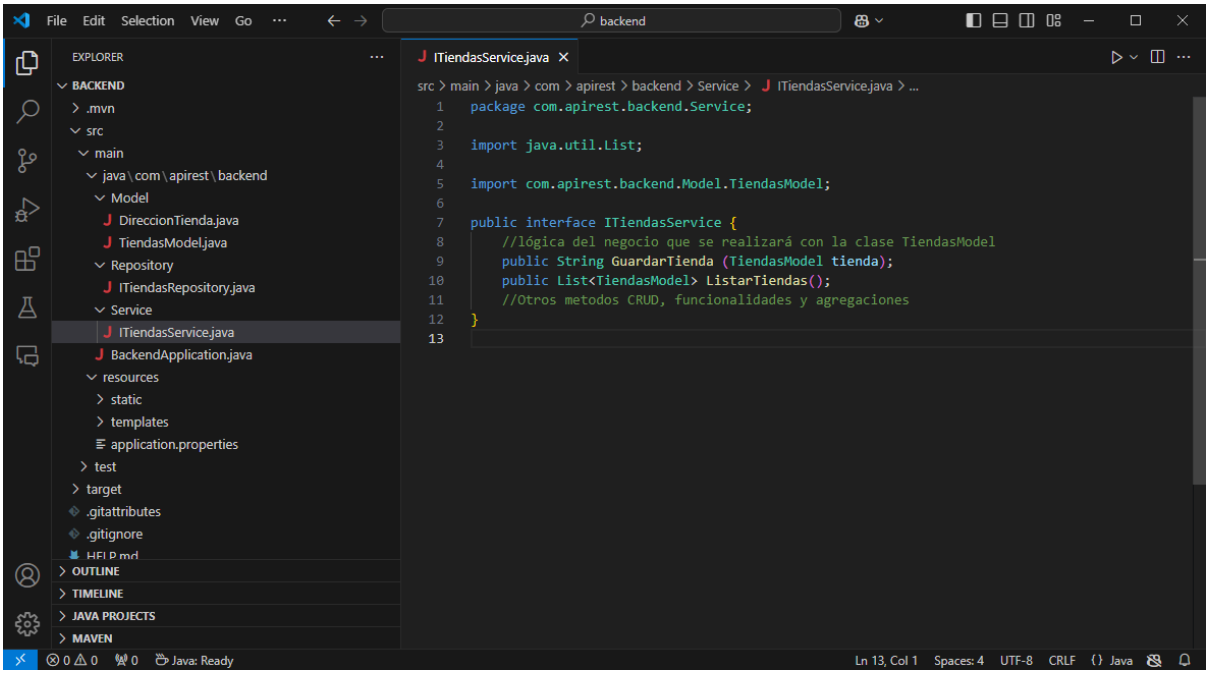

Figura 24. Interfaz ITiendasService

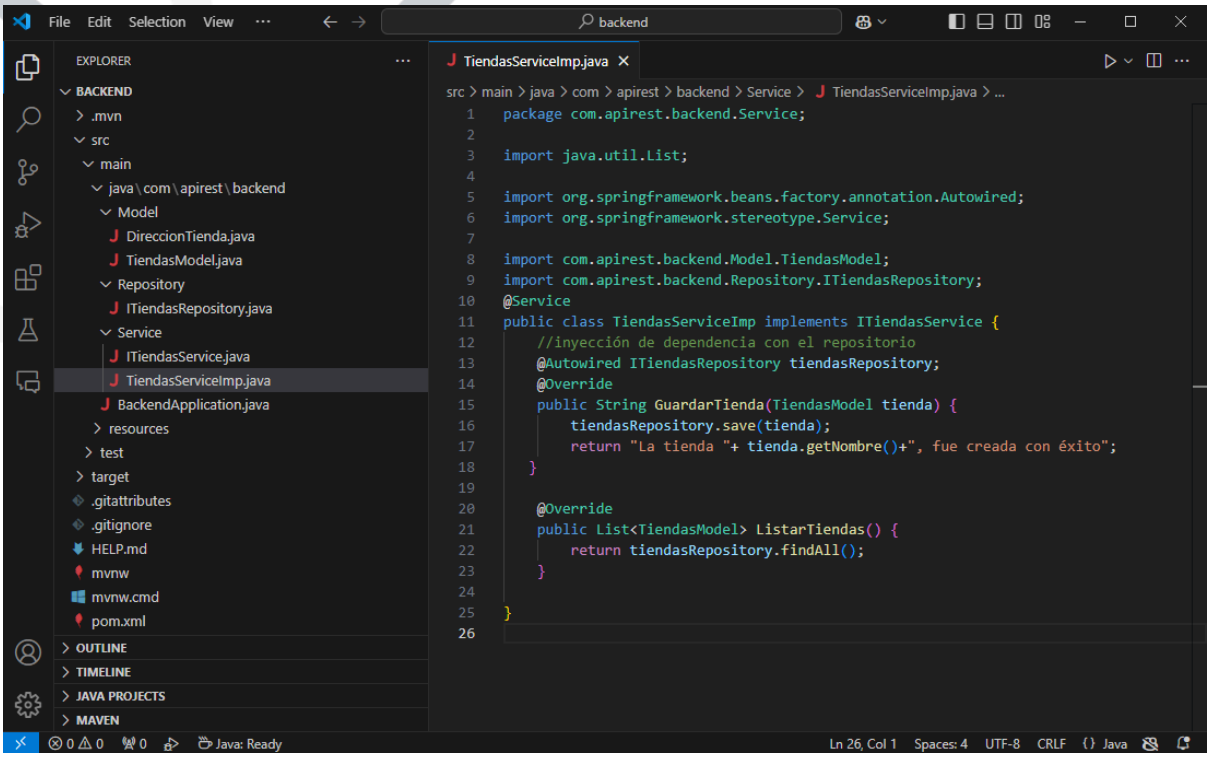

Figura 25. Clase TiendasServiceImp

Nota: En el contexto de la programación, una "dependencia" se refiere a un objeto o componente que otro objeto necesita para llevar a cabo su trabajo. Una correcta gestión de las dependencias permitirá garantizar la modularidad y mantenimiento del código.

Paso 4.4: Crear la capa "Controller".

Crear la carpeta "Controller" en la ruta del proyecto "java/com/apirest/backend", y dentro de esta crear la clase "TiendasController". Para identificar que esta clase representa un controlador en el proyecto, se debe utilizar la anotación *"@RestController"*. Esta anotación permite exponer servicios web en el desarrollo de aplicaciones RESTful, facilitando la interacción con el backend a través de formatos como JSON.

Se deberá utilizar la anotación "@RequestMapping" para especificar la URL (Endpoint) del controlador, que definirá el punto de acceso para interactuar con las funcionalidades definidas para la clase "TiendasModel". Por ejemplo, "/UAO/apirest/Tiendas".

Dentro de la clase "TiendasController" se implementarán los métodos que permitirán gestionar las solicitudes HTTP entrantes. En este sentido se deberá realizar una inyección de dependencias con la interfaz "ITiendasService" para generar la comunicación haciendo uso de la anotación "@Autowired", lo que permitirá establecer la comunicación con la capa de servicio.

Además, se implementarán los siguientes métodos:

- Crear tienda que estará asociado a la anotación "@PostMapping" y permitirá gestionará las solicitudes para crear nuevos documentos en la base de datos.
- Listar tiendas que estará asociado a la anotación "@GetMapping" y permitirá recuperar todos los documentos de la colección Tiendas de la base de datos.

Ambas anotaciones, incluirán una subruta específica que al combinarlas con la URL del Endpoint conformarán las rutas finales de los servicios desarrollados. La implementación completa de este componente se puede observar en la Figura 25.

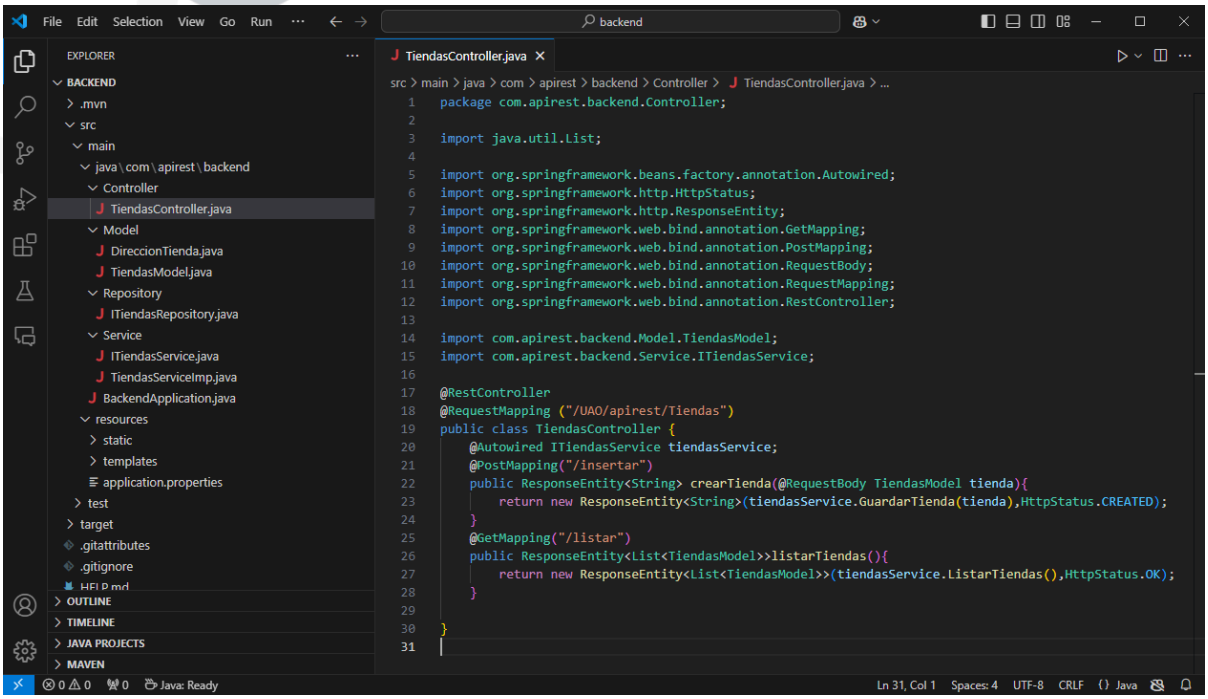

Figura 25. Clase TiendasController

Paso 5: Configurar la Conexión del Proyecto con la Base de Datos

Para establecer la conexión con MongoDB es necesario configurar el archivo “application.properties”. Este archivo se encuentra ubicado en la carpeta “resources”, como se puede observar en la Figura 26.

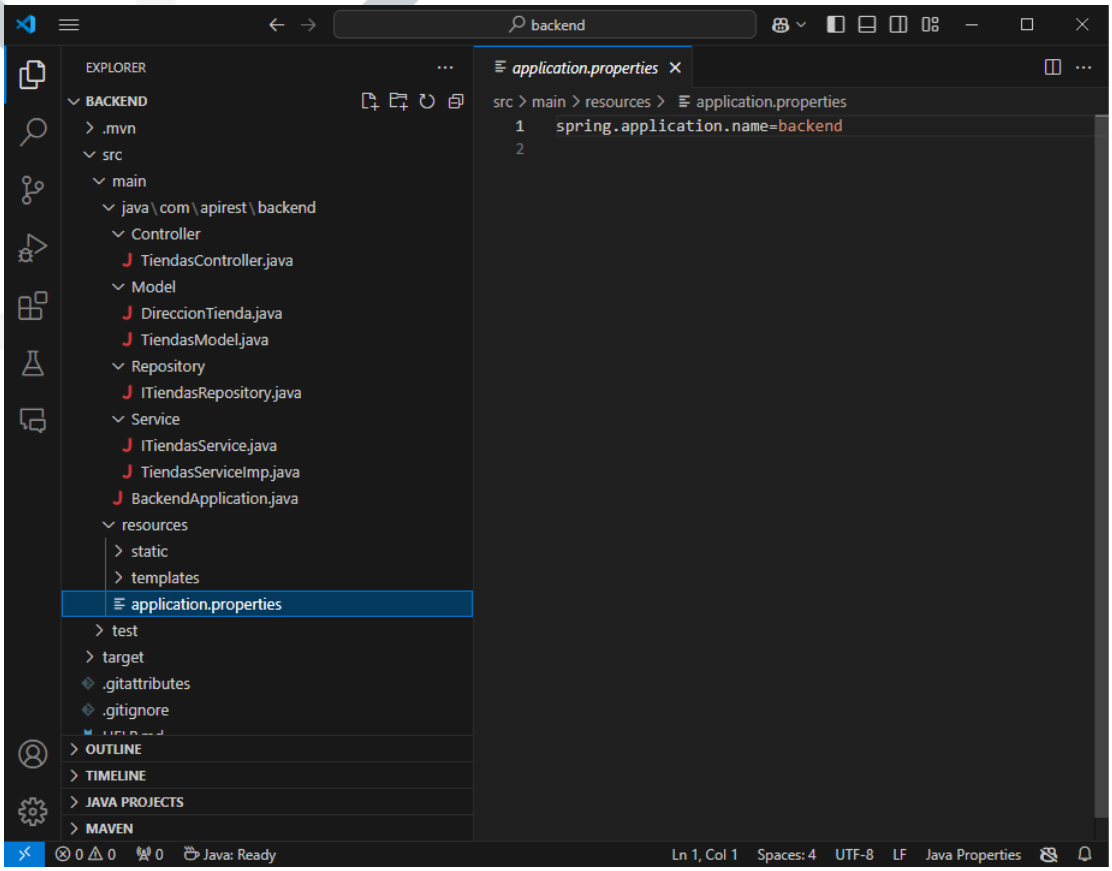

Figura 26. Archivo application.properties

En el archivo “application.properties”, se deberán definir los siguientes parámetros de conexión:

- **Host**: Dirección del servidor donde está alojada la base de datos MongoDB.
- **Puerto**: Número del puerto utilizado para establecer la conexión con MongoDB.
- **Nombre de la base de datos**: Nombre exacto de la base de datos que se usará en el proyecto.

La configuración completa de este archivo se muestra en la Figura 27. Esta configuración es fundamental para que el proyecto Spring Boot pueda interactuar correctamente con la base de datos.

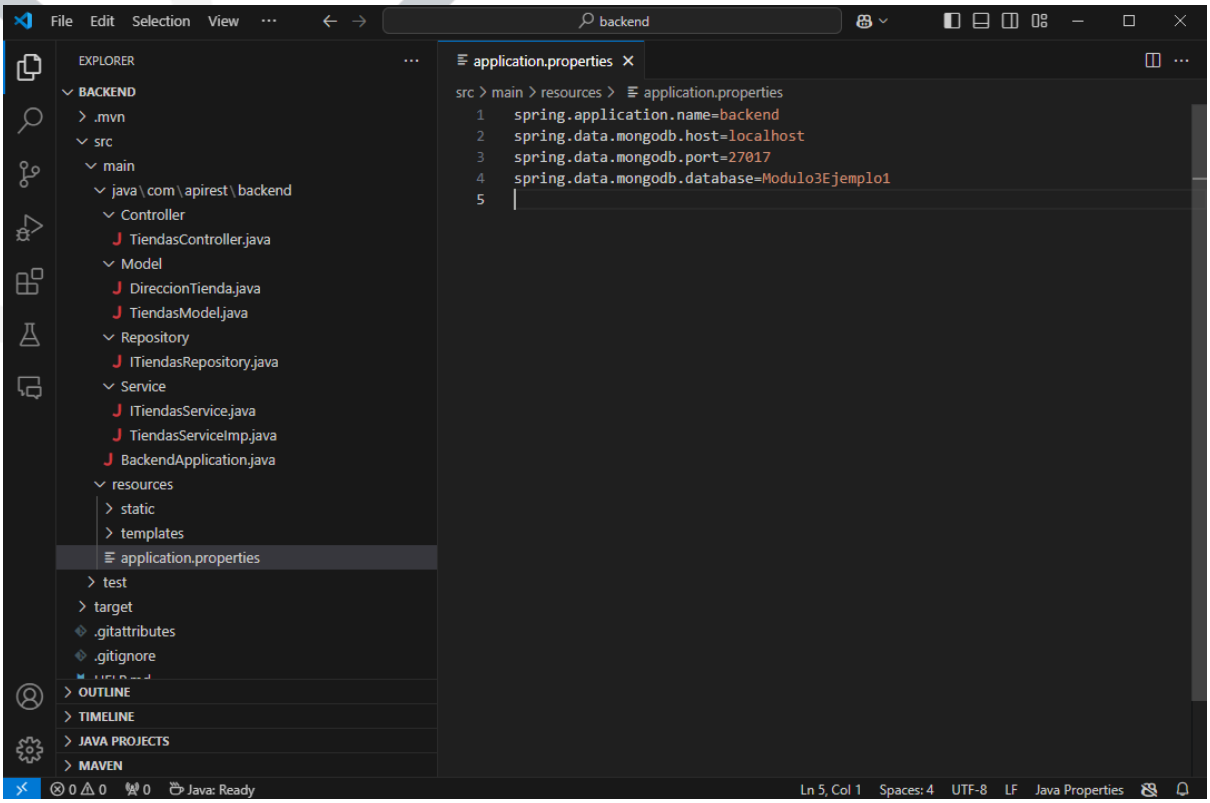

Figura 27. Configuración de la Conexión del Proyecto con MongoDB

Paso 6: Ejecutar el Proyecto.

Para iniciar la ejecución del backend, primero se deberá localizar la clase que contiene el método principal del proyecto. Esta clase se encuentra en la ruta java/com/apirest/backend y su nombre corresponde al nombre del proyecto seguido de la palabra Application.

Una vez localizada, haga clic en la opción Run para ejecutar el programa principal. Si el proyecto ha sido configurado correctamente, al final de la consola deberá aparecer un mensaje similar a:
“Started BackendApplication in 3.299 seconds (process running for 3.775)”

Este mensaje indicará que el backend se ha iniciado correctamente y está en ejecución. En la Figura 28 se puede observar el resultado de la ejecución del programa principal del backend.

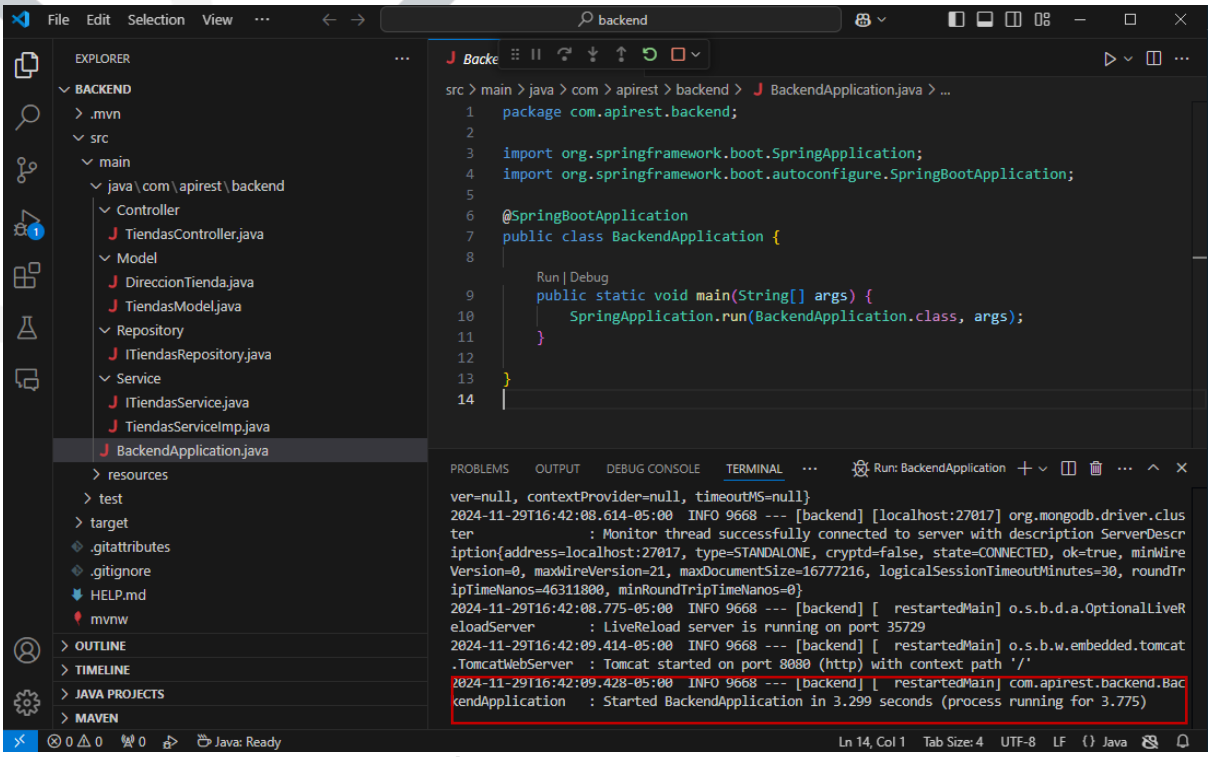

Figura 28. Ejecución del Programa Principal del Backend

Paso 7: Pruebas funcionales al Backend

Para realizar las pruebas a las funcionalidades implementadas en el backend, se deberá abrir Postman y dar clic en el icono "+" que hace referencia a "Create new Collection" y dar el nombre de Tiendas. Luego se deberá crear una petición HTTP para poder agregar una tienda a la base de datos. Por tanto, dar clic en los "…" que hay junto al nombre de la colección y en el menú alternativo dar clic en "Add request", como se observa en la Figura 29.

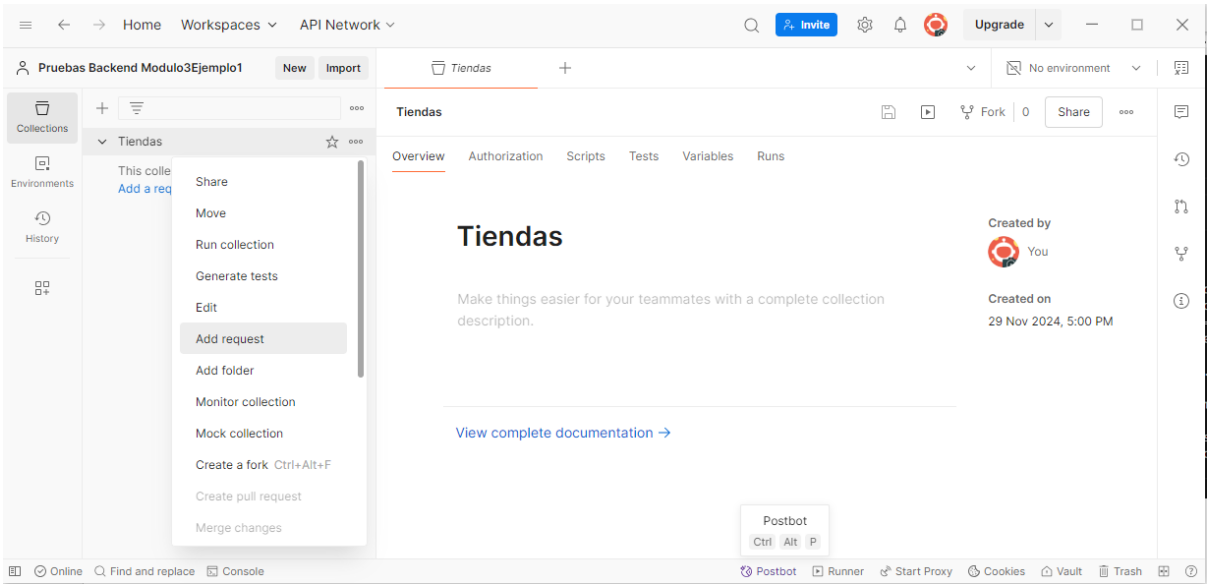

Figura 29. Postman – Adicionar una Petición HTTP

Una vez se adicionada la petición o solicitud HTTP se abrirá una nueva ventana (Ver Figura 30) donde se deberá especificar el método, formato y el cuerpo.

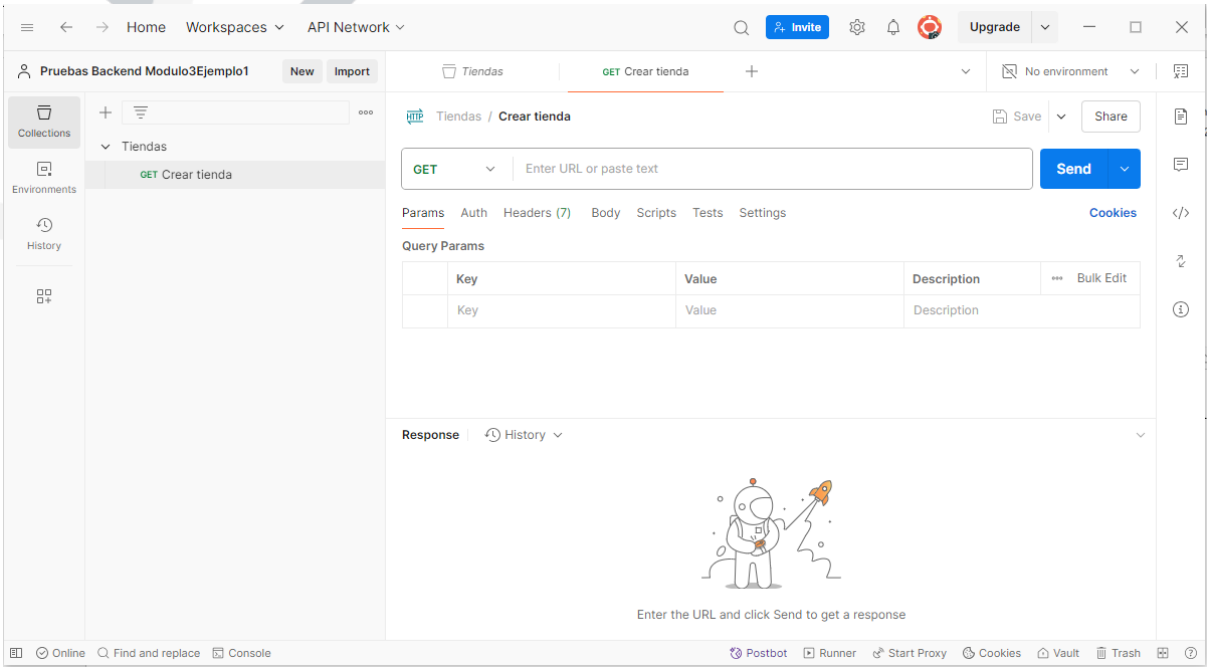

Figura 30. Postman – Ventana de la petición

Ahora, se deberá especificar el método de la petición HTTP que al estar asociada al método crear tiendas que está definido en el controlador, deberá ser POST. Se deberá fijar la URL asociada a esta petición la cual deberá ser "http://localhost:8080/UAO/apirest/Tiendas/insertar". Se establece el formato, para esto, se deberá ir a "Body", escoger "raw" y en "text" especificar que el formato es "JSON". En el área de trabajo se deberá detallar el documento JSON con la información de la tienda. Una vez creado el documento JSON, dar clic en el botón "Send" para realizar el envío de la petición. Si la petición es correcta se deberá mostrar el mensaje "La tienda Tienda TecnoSoftware, fue creada con éxito" y el código "201 Created" (Ver Figura 31).

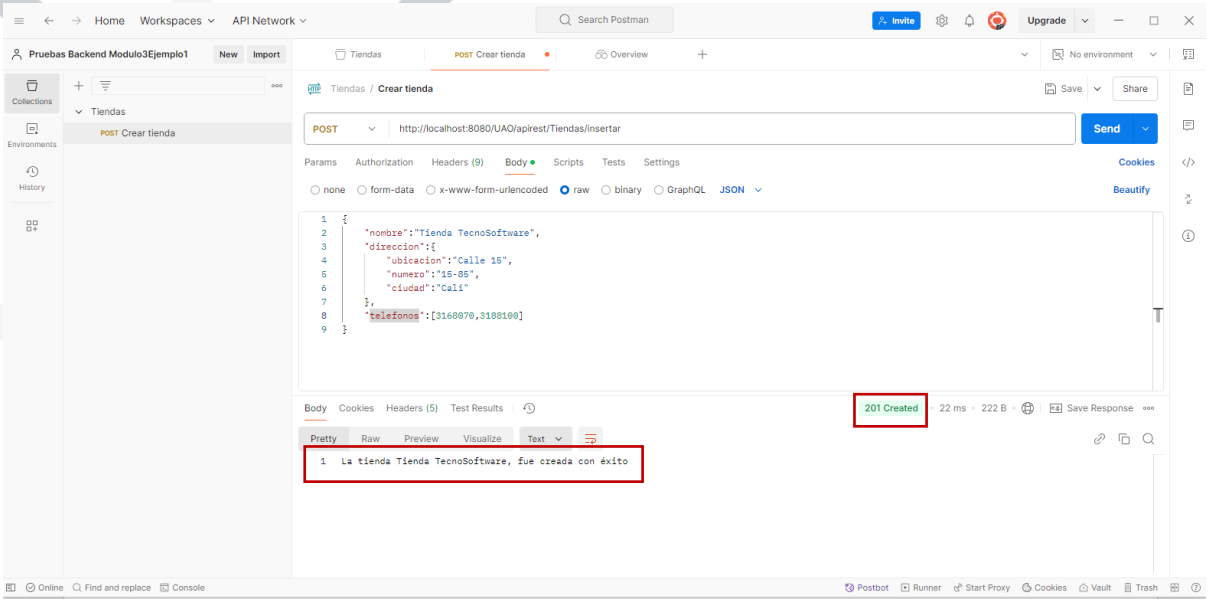

Figura 31. Postman – Petición Crear Tienda Completa

Finalmente, para listar todas las tiendas que se encuentran registradas en la base de datos, se deberá crear una petición GET que estará asociada la siguiente URL que se configuro en controlador "http://localhost:8080/UAO/apirest/Tiendas/listar". El resultado de esta petición se puede observar en la Figura 32. Si la petición es correcta se retornará un arreglo de documentos JSON con las tiendas creadas y el código HTML "200 OK".

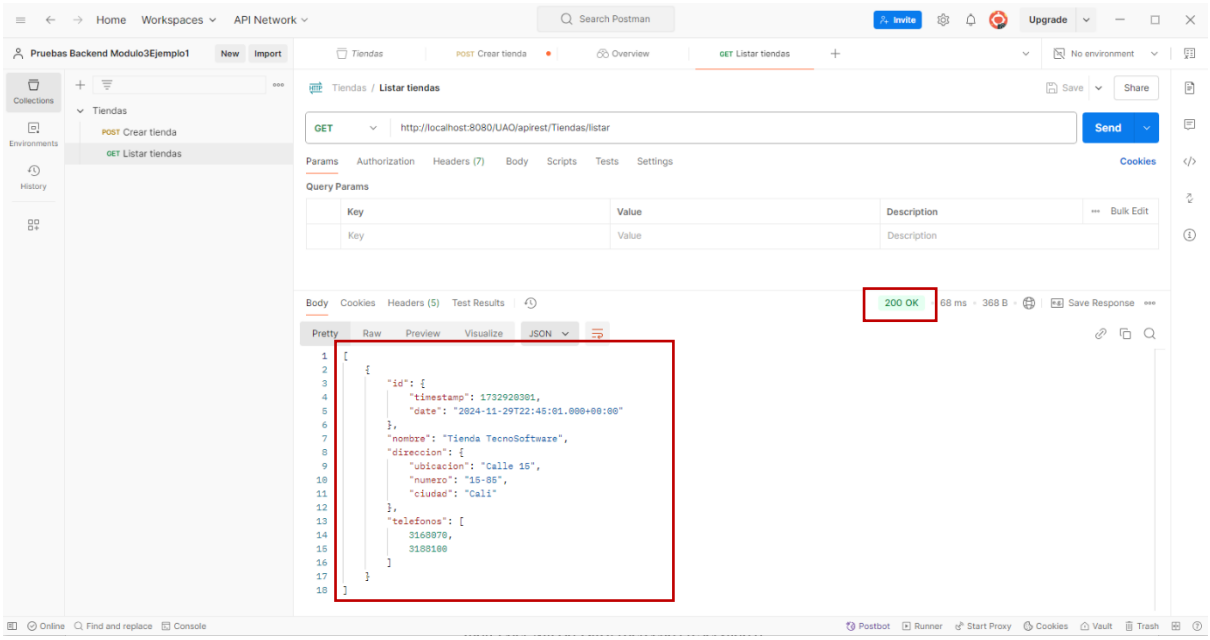

Figura 32. Postman – Envío de la Petición Listar Tiendas

En la figura 32, se puede observar que el id del documento no se está mostrando como un ObjectId sino como una marca de tiempo (timestamp). Si se desea mostrar el valor real del ObjetId en hexadecimal que fue asignado por MongoDB, se deberá agregar el siguiente método en la clase "TiendasModel"

```
@JsonProperty("id")
   public String getIdAsString() {
       return id != null ? id.toHexString() : null;
   }
```

Explicación: La anotación “@JsonProperty se utiliza para especificar el nombre del campo JSON que se desea mapear desde un atributo en la clase Java “TiendasModel” para brindar la respuesta. El método “getIdAsString()” permite convertir el valor del ObjectId (formato hexadecimal) del id a String haciendo uso del método “toHexString()”. Este método permitirá que el id se muestre como una cadena texto que sea legible y útil al serializar el objeto en JSON. En la Figura 33, se muestra la implementación de este método, y en la Figura 34 se ilustra su funcionamiento durante una consulta.

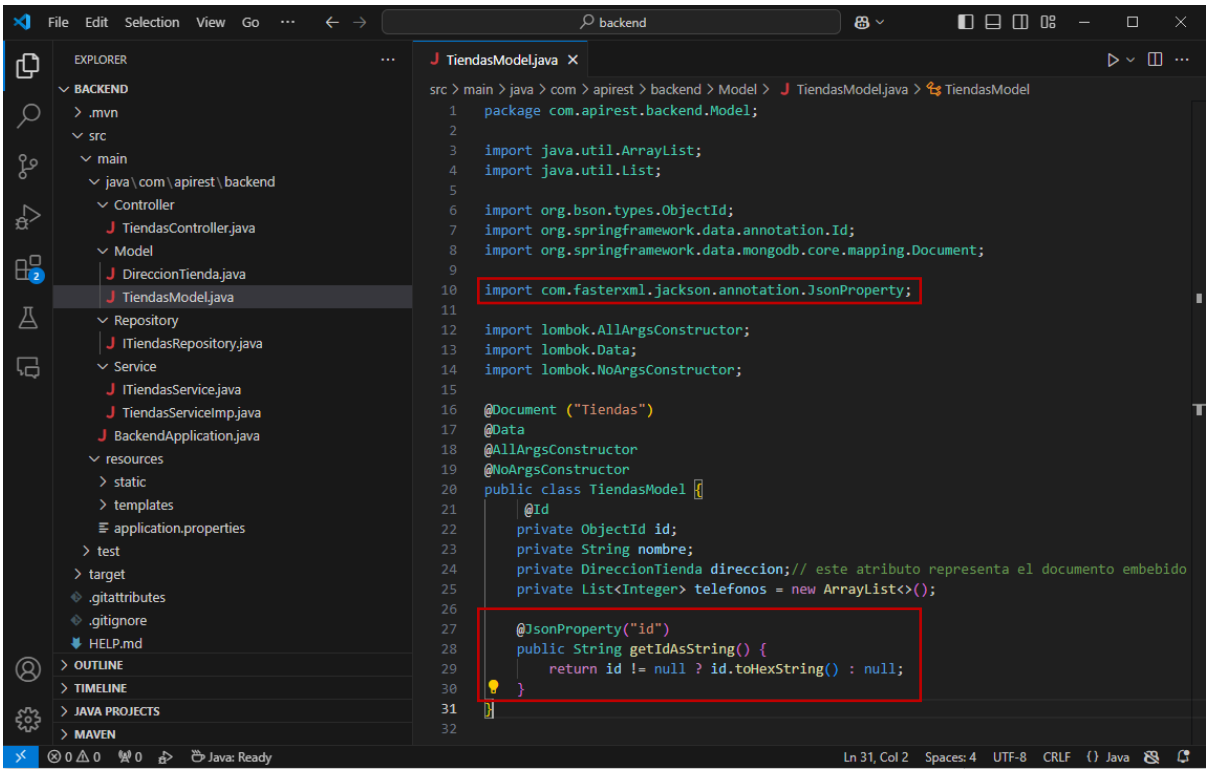

Figura 33. Actualización de la Clase TiendasModel

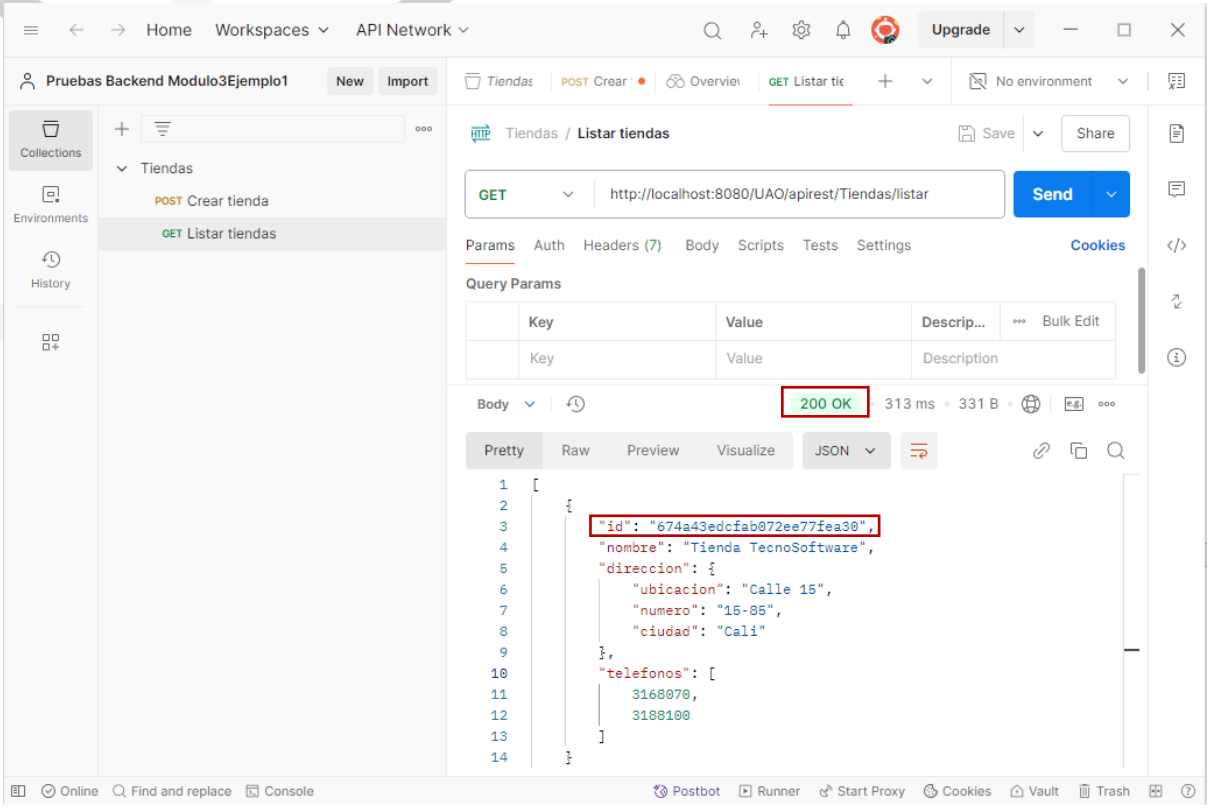

Figura 34. Postman – Envío 2 de la Petición Listar Tiendas

Paso 8: Otras operaciones CRUD y consulta personalizada

Ahora se implementarán las funcionalidades de buscar una tienda por id, actualizar y eliminar una tienda. Adicionalmente, se implementará la consulta listar todas las tiendas pertenecientes a una ciudad.

Paso 8.1: Crear la capa "Exception".

Esta capa se encargará de centralizar y organizar el manejo de error en la aplicación mediante excepciones personalizadas que representen errores específicos, lo que permitirá separar el manejo del error de la lógica del negocio para hacer el código más limpio. En este sentido, esta capa contendrá las siguientes clases:

- **RecursoNoEncontradoException**: Esta clase representa una excepción personalizada que permite lanzar un error al no encontrar un recurso específico en la aplicación. Por ejemplo, al realizar una búsqueda por id. La implementación de esta clase se presenta en la Figura 35.

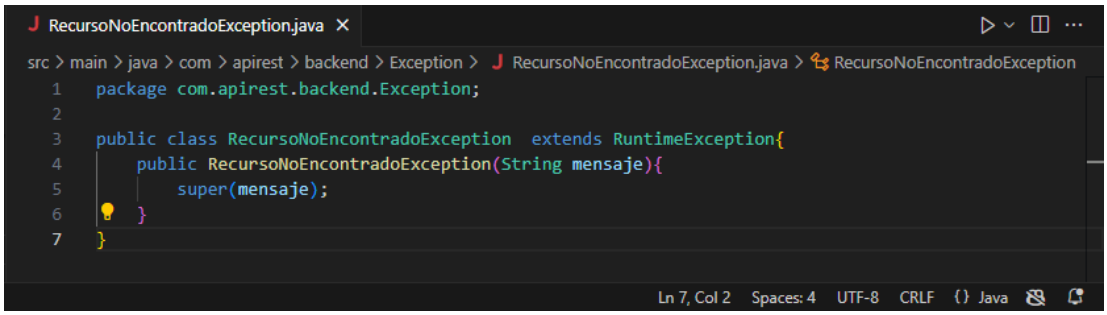

Figura 35. Clase RecursoNoEncontradoException

- **GlobalExceptionHandler**: Esta clase centraliza la capturara y gestión excepciones de la aplicación, permitiendo formatear las salidas de los errores para que sean más claras para los clientes. Es decir, se mostrará únicamente el mensaje de error personalizado que fuer asociado a una funcionalidad en la aplicación y el código HTML correspondiente su implementación se puede apreciar en la Figura 36.

```
GlobalExceptionHandler.java
src > main > java > com > apirest > backend > Exception > GlobalExceptionHandler.java > ...
1   package com.apirest.backend.Exception;
2
3   import java.util.HashMap;
4   import java.util.Map;
5
6   import org.springframework.http.HttpStatus;
7   import org.springframework.http.ResponseEntity;
8   import org.springframework.web.bind.annotation.ExceptionHandler;
9   import org.springframework.web.bind.annotation.RestControllerAdvice;
10
11  @RestControllerAdvice
12  public class GlobalExceptionHandler{
13      @ExceptionHandler(RecursoNoEncontradoException.class)
14      public ResponseEntity<Map<String, String>> manejarRecursoNoEncontrado(RecursoNoEncontradoException ex) {
15          Map<String, String> errorResponse = new HashMap<>();
16          errorResponse.put(key:"error", ex.getMessage());
17          return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
18      }
19
20      @ExceptionHandler(Exception.class)
21      public ResponseEntity<Map<String, String>> manejarErroresGenerales(Exception ex) {
22          Map<String, String> errorResponse = new HashMap<>();
23          errorResponse.put(key:"error", value:"Ocurrió un error inesperado. Por favor, intente más tarde.");
24          return new ResponseEntity<>(errorResponse, HttpStatus.INTERNAL_SERVER_ERROR);
25      }
26
27  }
28
Ln 28, Col 1   Spaces: 4   UTF-8   CRLF   {} Java
```

Figura 36. Clase GlobalExceptionHandler

Paso 8.2: Crear la capa "DTO".

Esta capa se encargará de ayudar a mejorar la estructura y la eficiencia de la aplicación. Esta capa contendrá la clase "TiendaDTO", la cual transformará y formateará los datos que se mostrarán como resultado de una consulta. Esto con el fin de proteger la estructura interna de la base de datos y proporcionar flexibilidad a la aplicación para presentar los datos de forma adecuada. En este sentido, la clase "TiendaDTO" se implementó específicamente para mostrar los valores necesarios que proyectará la consulta "listar todas las tiendas pertenecientes a una ciudad", ya que la clase "TiendasModel" no contiene la misma estructura de los campos que proyecta la consulta.

```java
TiendaDTO.java
src > main > java > com > apirest > backend > DTO > TiendaDTO.java > TiendaDTO
package com.apirest.backend.DTO;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class TiendaDTO {
    private String nombre;
    private String direccionCompleta;
}
Ln 13, Col 2  Spaces: 4  UTF-8  CRLF  {} Java
```

Figura 37. Clase TiendaDTO

Nota: Si las consultas proyectan un conjunto de campos que son diferente al de la estructura interna de una colección, se deberá crear un DTO para mapear y presentar los datos de forma correcta.

Paso 8.3: Actualizar la Interfaz ITiendasRepository.
Ahora se deberá actualizar la interfaz "ITiendasRepository" para definir el pipeline para la consulta requerida utilizando la anotación "@Aggregation". Además, se tendrá que definir el método "buscarTiendasPorCiudad" para poder capturar los valores que arroja la consulta ejecutada en MongoDB. La implementación de esta consulta se puede observar en la Figura 38.

```java
ITiendasRepository.java
src > main > java > com > apirest > backend > Repository > ITiendasRepository.java > ITiendasRepository
package com.apirest.backend.Repository;

import java.util.List;

import org.bson.types.ObjectId;
import org.springframework.data.mongodb.repository.Aggregation;
import org.springframework.data.mongodb.repository.MongoRepository;

import com.apirest.backend.DTO.TiendaDTO;
import com.apirest.backend.Model.TiendasModel;

public interface ITiendasRepository extends MongoRepository<TiendasModel, ObjectId> {
    //Aquí se implementarán las agregaciones
    @Aggregation(pipeline = {
        "{ $match: { 'direccion.ciudad': ?0 } }",
        "{ $project: { _id: 0, nombre: 1, "+
                    "direccionCompleta: { "+
                            "$concat: [ '$direccion.ubicacion',' ','$direccion.numero'] "+
                        "}"+
                    "}"+
        "}"
    })
    List<TiendaDTO> buscarTiendasPorCiudad(String ciudad);
}
Ln 24, Col 2  Spaces: 4  UTF-8  CRLF  {} Java
```

Figura 38. Actualización ITiendasRepository

Nota: *El "?0" en la etapa $match hace referencia al primer parámetro del método "buscarTiendasPorCiudad", el* cual tomará el valor una vez se realice el llamado a la consulta.

Paso 8.4: Actualizar el servicio

En este paso, se deberá actualizar la lógica del negocio a través de la implementación de los siguientes métodos:

- Buscar una tienda por su identificador (id).
- Actualizar los datos de una tienda existente.
- Eliminar una tienda específica.
- Listar todas las tiendas ubicadas en una ciudad determinada.

En la Figura 39 se presenta la actualización a la interfaz “ITiendasService”, mientras que la implementación de la lógica del negocio en la clase “TiendasServiceImp” puede ser observada en la Figura 40.

ITiendasService.java

src > main > java > com > apirest > backend > Service > ITiendasService.java > ITiendasService

```java
package com.apirest.backend.Service;

import java.util.List;

import org.bson.types.ObjectId;

import com.apirest.backend.DTO.TiendaDTO;
import com.apirest.backend.Model.TiendasModel;

public interface ITiendasService {
    //lógica del negocio que se realizará con la clase TiendasModel
    public String GuardarTienda (TiendasModel tienda);
    public List<TiendasModel> ListarTiendas();
    //Otros metodos CRUD, funcionalidades
    public TiendasModel buscarTiendaPorId(ObjectId id);
    public TiendasModel actualizarTienda(ObjectId id, TiendasModel tienda);
    public String eliminarTienda (ObjectId id);
    //agregaciones
    List<TiendaDTO> buscarTiendasPorCiudad(String ciudad);
}
```

Ln 20, Col 2 Spaces: 4 UTF-8 CRLF {} Java

Figura 39. Actualización ITiendasService

```java
TiendasServiceImp.java
src > main > java > com > apirest > backend > Service > TiendasServiceImp.java > TiendasServiceImp > buscarTiendaPorId(ObjectId)
14  public class TiendasServiceImp implements ITiendasService {
27
28      @Override
29      public TiendasModel buscarTiendaPorId(ObjectId id) {
30          // Este método utiliza una función lambda para buscar una tienda por su ID en el repositorio.
31          // Si la tienda existe, retorna el objeto correspondiente. De lo contrario, lanza una excepción
32          // personalizada RecursoNoEncontradoException con un mensaje descriptivo.
33          return tiendasRepository.findById(id).
34          orElseThrow(() -> new RecursoNoEncontradoException("Error!. No existe una tienda con ID: " + id +" o está mal escrito"));
35      }
36
37      @Override
38      public TiendasModel actualizarTienda(ObjectId id, TiendasModel tienda) {
39          // Usamos el método buscarTiendaPorId para garantizar que la tienda exista en la base de datos
40          TiendasModel tiendaExistente = buscarTiendaPorId(id);
41          // Actualizar los valores de la tienda si y solo si existente
42              tiendaExistente.setNombre(tienda.getNombre());
43              tiendaExistente.setDireccion(tienda.getDireccion());
44              tiendaExistente.setTelefonos(tienda.getTelefonos());
45
46          // Guardar la tienda actualizada
47          return tiendasRepository.save(tiendaExistente);
48      }
49
50      @Override
51      public String eliminarTienda(ObjectId id) {
52          // Usamos el método buscarTiendaPorId para garantizar que la tienda exista en la base de datos
53          TiendasModel tiendaExistente = buscarTiendaPorId(id);
54
55          // Eliminar la tienda
56          tiendasRepository.delete(tiendaExistente);
57          // Retornamos un mensaje de confirmación de la acción realziada
58          return "La tienda con ID " + id + " fue eliminada correctamente.";
59      }
60
61      @Override
62      public List<TiendaDTO> buscarTiendasPorCiudad(String ciudad) {
63         return tiendasRepository.buscarTiendasPorCiudad(ciudad);
64      }
65  }
Ln 32, Col 9   Spaces: 4   UTF-8   CRLF   {} Java
```

Figura 40. Actualización TiendasServiceImp

Paso 8.5: Actualizar el controlador TiendasController

En este paso, se deberá actualizar el controlador “TiendasController” para que los usuarios de la aplicación puedan interactuar con la nueva lógica del negocio desarrollada. Por tanto, será necesario crear las rutas correspondientes para los siguientes métodos:

- **Consultar datos**: Para los métodos buscarTiendaPorId y ListarTiendasPorCiudad, se utilizará la anotación “@GetMapping”, especificando la ruta y la variable de entrada entre llaves {nombreVariable}.
- **Actualizar datos**: Para el método de actualización, se empleará la anotación “@PutMapping”, indicando la ruta y la variable de entrada correspondiente.
- **Eliminar datos**: Para el método de eliminación, se utilizará la anotación “@DeleteMapping” con su respectiva ruta y variable de entrada.

La actualización de este componente se detalla en la Figura 41.

TiendasController.java

src > main > java > com > apirest > backend > Controller > TiendasController.java > TiendasController

```java
package com.apirest.backend.Controller;

import java.util.List;

import org.bson.types.ObjectId;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.apirest.backend.DTO.TiendaDTO;
import com.apirest.backend.Model.TiendasModel;
import com.apirest.backend.Service.ITiendasService;

@RestController
@RequestMapping ("/UAO/apirest/Tiendas")
public class TiendasController {
    @Autowired ITiendasService tiendasService;
    @PostMapping("/insertar")
    public ResponseEntity<String> crearTienda(@RequestBody TiendasModel tienda){
        return new ResponseEntity<String>(tiendasService.GuardarTienda(tienda),HttpStatus.CREATED);
    }
    @GetMapping("/listar")
    public ResponseEntity<List<TiendasModel>>listarTiendas(){
        return new ResponseEntity<List<TiendasModel>>(tiendasService.ListarTiendas(),HttpStatus.OK);
    }
    @GetMapping("/TiendaPorId/{id}")
    public ResponseEntity<TiendasModel> buscarTiendaPorId(@PathVariable ObjectId id){
        return new ResponseEntity<TiendasModel>(tiendasService.buscarTiendaPorId(id),HttpStatus.OK);
    }
    @PutMapping("/actualizar/{id}")
    public ResponseEntity<TiendasModel> actualizarTienda(@PathVariable ObjectId id, @RequestBody TiendasModel tienda) {
        return new ResponseEntity<TiendasModel>(tiendasService.actualizarTienda(id, tienda),HttpStatus.OK);
    }

    @DeleteMapping("/eliminar/{id}")
    public ResponseEntity<String> eliminarTienda(@PathVariable ObjectId id) {
        return new  ResponseEntity<String>(tiendasService.eliminarTienda(id),HttpStatus.OK);
    }
    @GetMapping("/tiendas/ciudad/{ciudad}")
    public ResponseEntity<List<TiendaDTO>> ListarTiendasPorCiudad(@PathVariable String ciudad){
        return new ResponseEntity<List<TiendaDTO>>(tiendasService.buscarTiendasPorCiudad(ciudad),HttpStatus.OK);
    }
}
```

Ln 51, Col 2 Spaces: 4 UTF-8 CRLF {} Java

Figura 41. Actualizar TiendasController

Paso 9: Actualizar la configuración de la conexión del proyecto con la base de datos

En este paso, se establecerá la conexión del proyecto con la base de datos en MongoDB Atlas. Para ello, será necesario abrir MongoDB Compass, conectarse al clúster en MongoDB Atlas y crear la base de datos “Modulo3Ejemplo1” utilizando el script definido en los pasos anteriores. Luego, se deberá configurar el archivo “application.properties” con la información del clúster, incluyendo el string de conexión, las credenciales de acceso, y el nombre de la base de datos. Un ejemplo de esta configuración se muestra en la Figura 42.

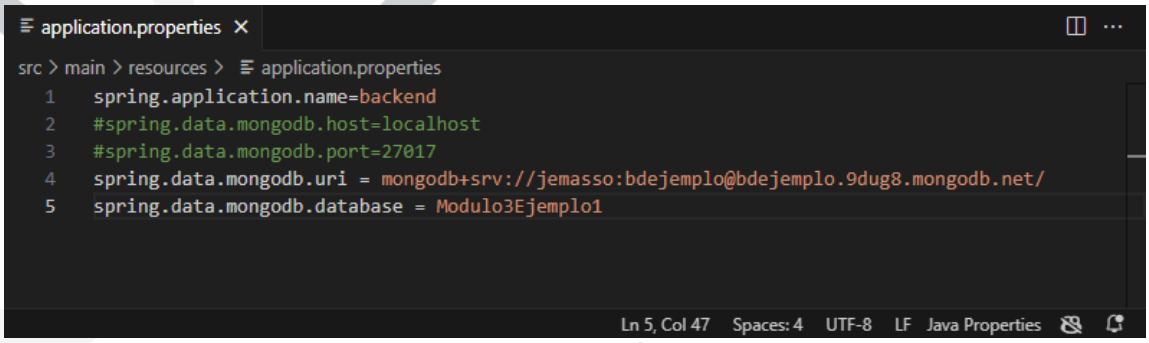

Figura 42. Actualizar la conexión de la base de datos

Paso 10: Nuevas pruebas funcionales al backend

En este paso, se realizarán nuevas pruebas funcionales y de integración a la base de datos "Modulo3Ejemplo1" ubicada en el clúster creado en MongoDB Atlas. A continuación, se describen los casos de prueba:

- **Caso 1**: Crear un nuevo documento.
  Se validará la integración mediante la creación de un nuevo documento en la colección "Tiendas".

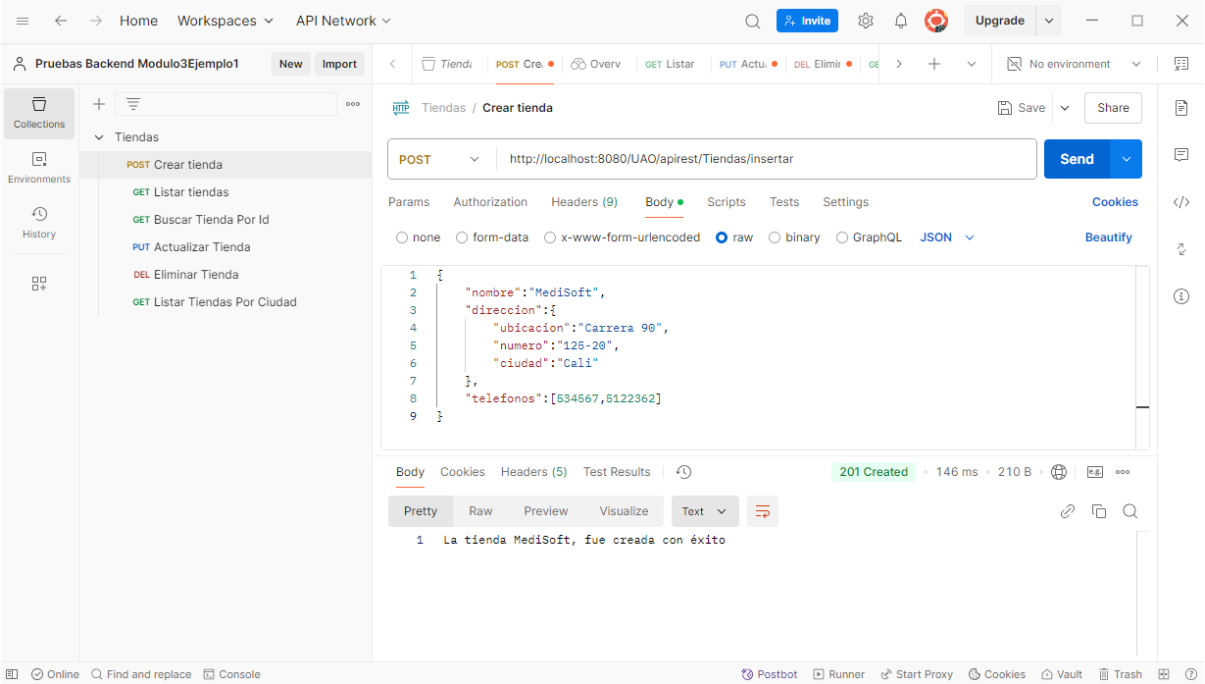

Figura 43. Crear una nueva tienda para validar la conexión

- **Caso 2**: Listar todos los documentos
  Se probará la funcionalidad para listar todos los documentos creados en la colección "Tiendas".

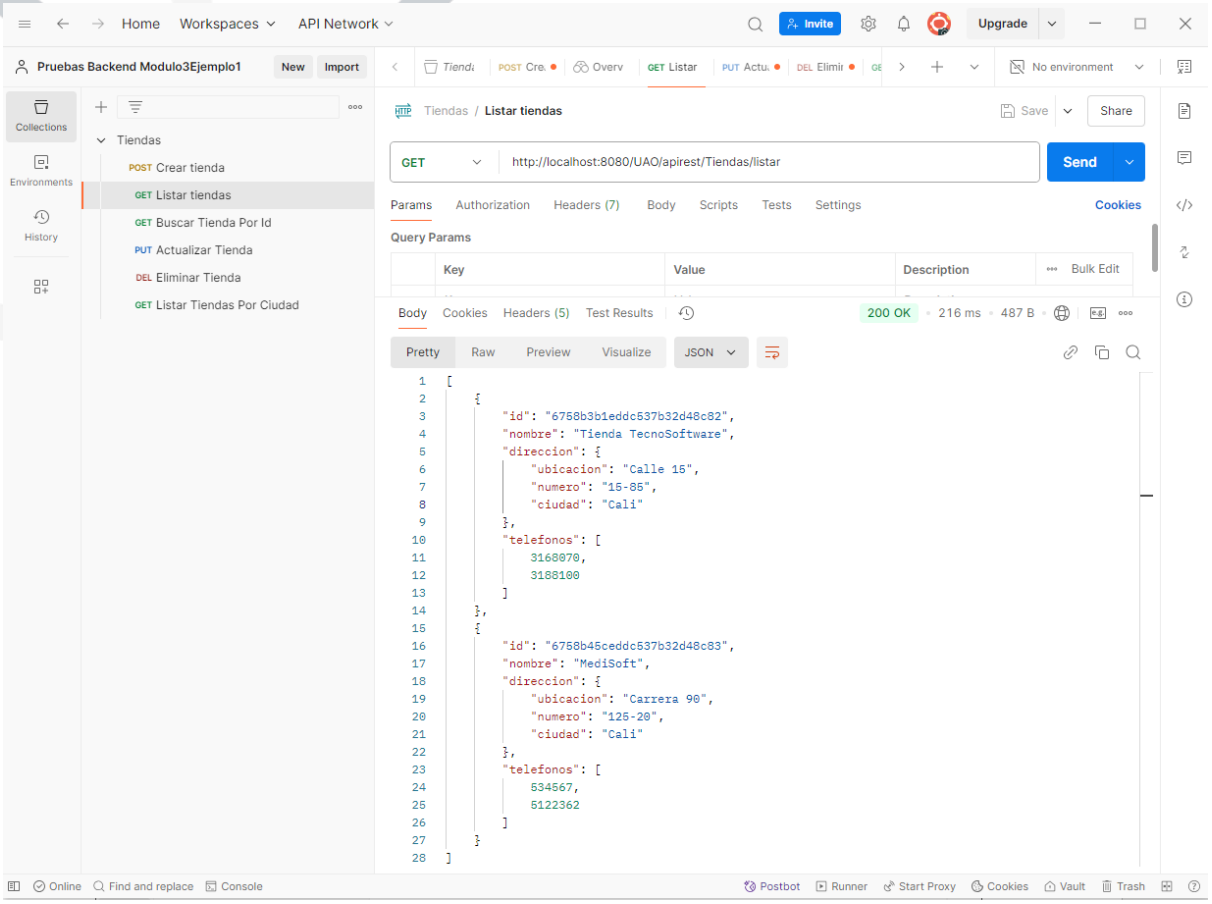

Figura 44. Listar Tiendas

- **Caso 3: Buscar una tienda por Id**
  - **Caso 3a (Éxito):** El id proporcionado existe en la base de datos.

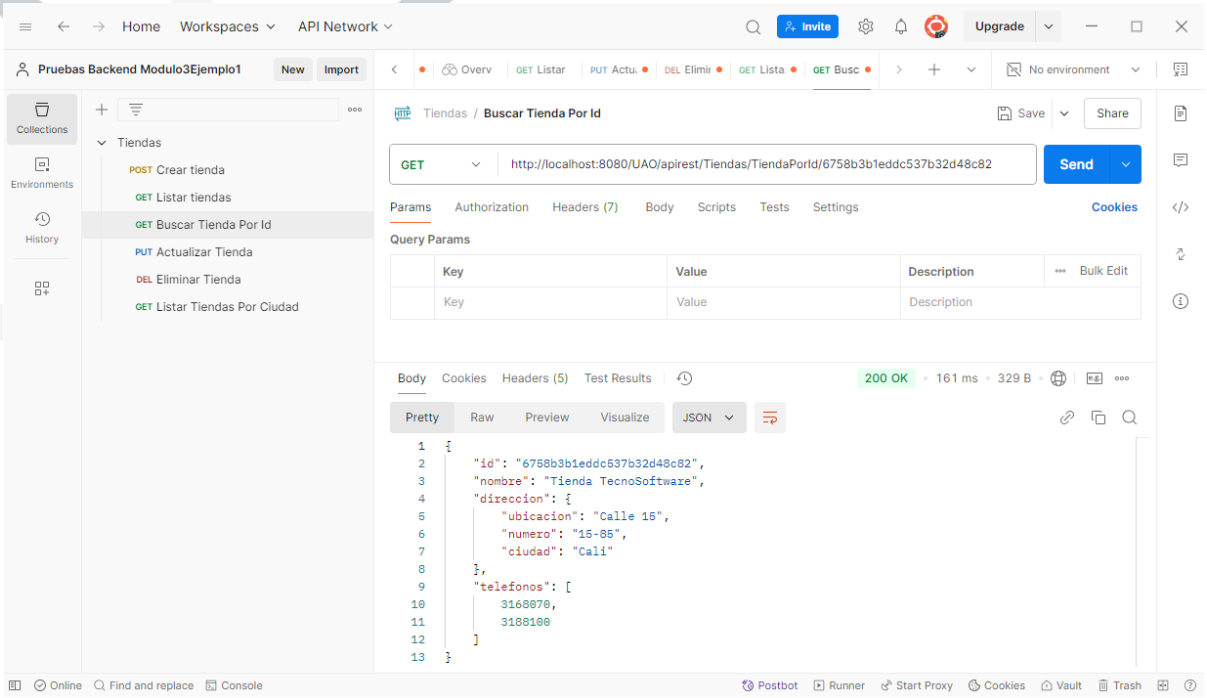

Figura 45. Buscar Tienda por Id – Caso de Éxito

- **Caso 3b (Error 1):** El id proporcionado no existe en la base de datos. Se deberá generar el mensaje de error definido en el método “buscarTiendaPorId”, utilizando la excepción personalizada “RecursoNoEncontradoException”.

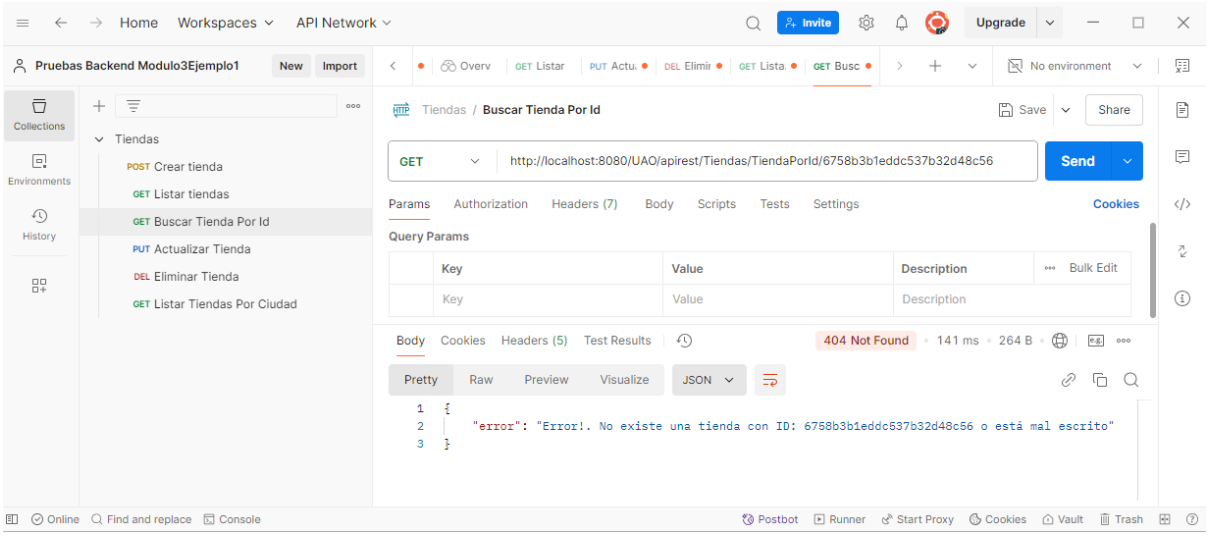

Figura 46. Buscar Tienda por Id – Caso de Erróneo 1

- **Caso 3c (Error 2):** No se proporciona un id para realizar la búsqueda. Se deberá mostrar el mensaje configurado en el manejador global de excepciones “GlobalExceptionHandler”.

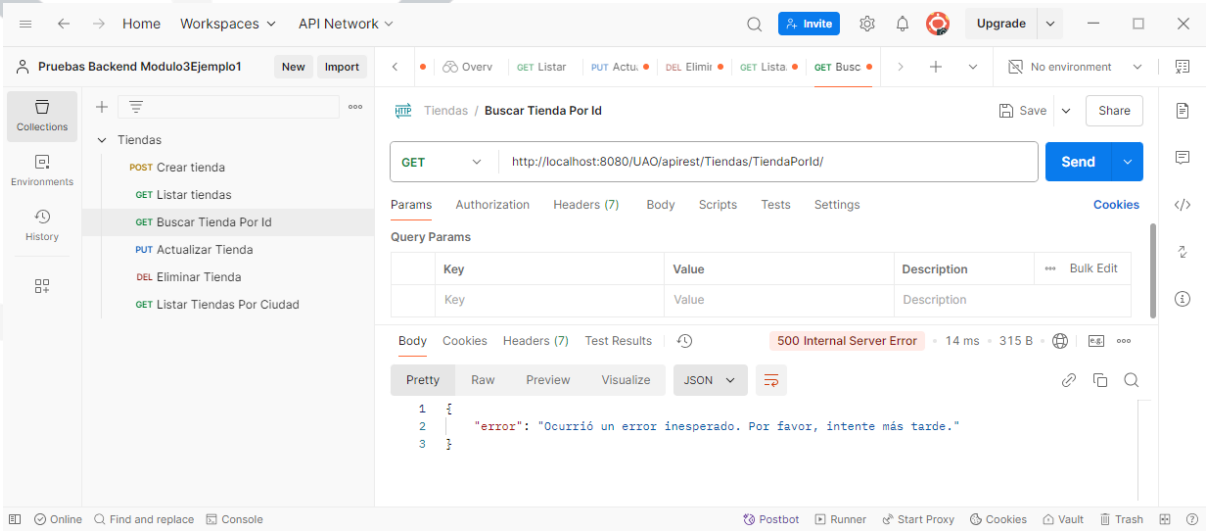

Figura 47. Buscar Tienda por Id – Caso de Erróneo 2

- **Caso 4: Actualizar una tienda por Id**
  - **Caso 4a (Éxito):** Se actualiza el nombre de una tienda y se agrega un nuevo teléfono.

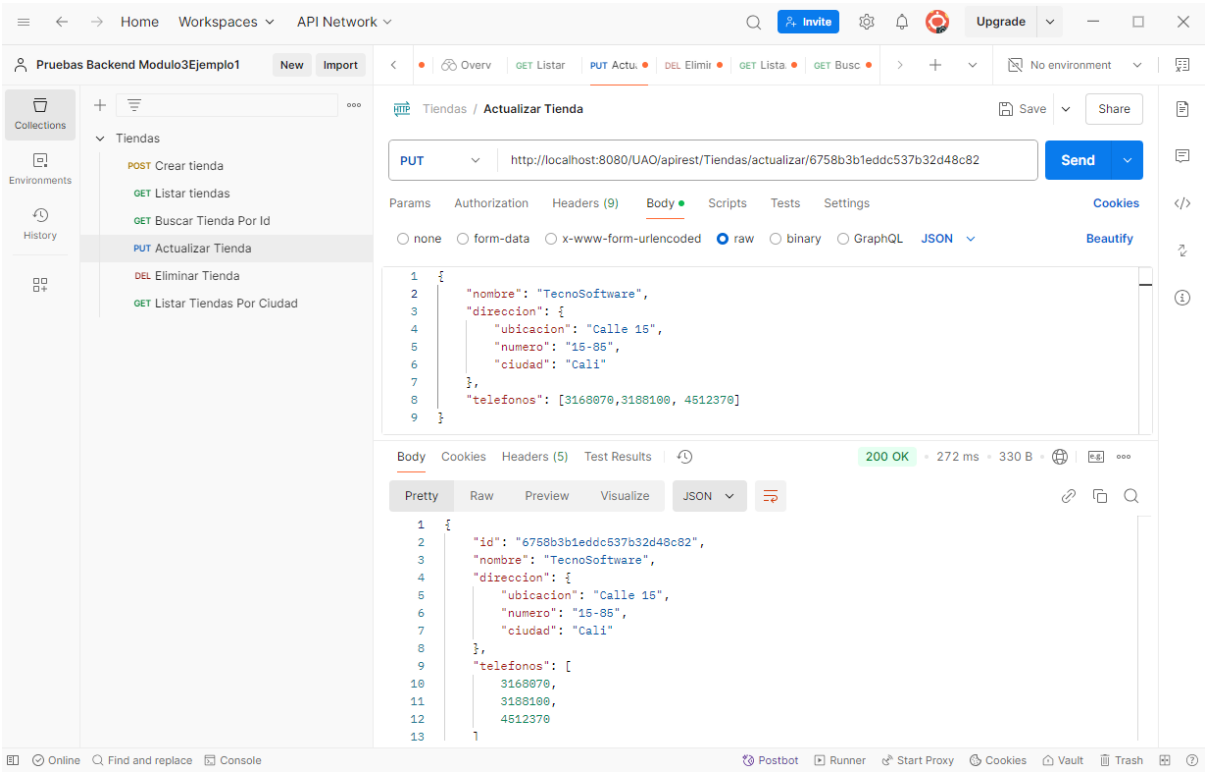

Figura 48. Actualizar Tienda por Id – Caso de Éxito

  - **Caso 4b (Error 1):** El id proporcionado no existe en la base de datos. Se deberá mostrar el mensaje de error definido en el método “buscarTiendaPorId”.

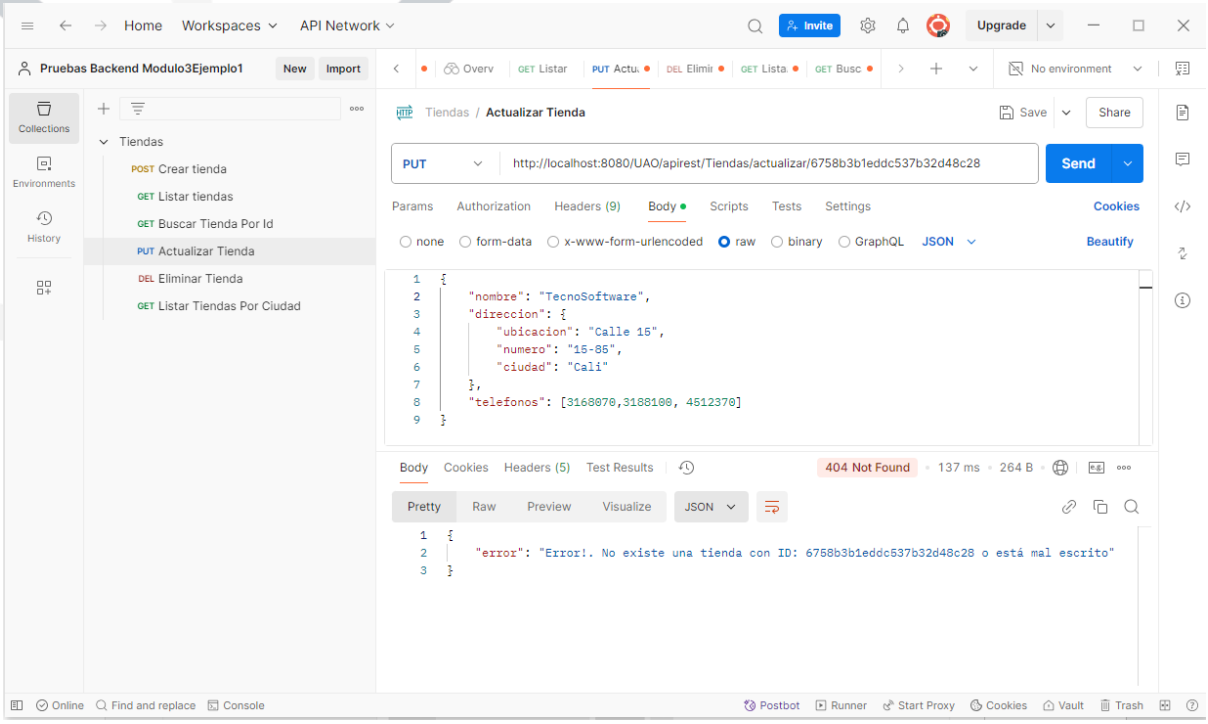

Figura 49. Actualizar Tienda por Id – Caso de Erróneo 1

- **Caso 4c (Error 2):** No se proporciona un id para realizar la actualización. El error será manejado por el “GlobalExceptionHandler”.

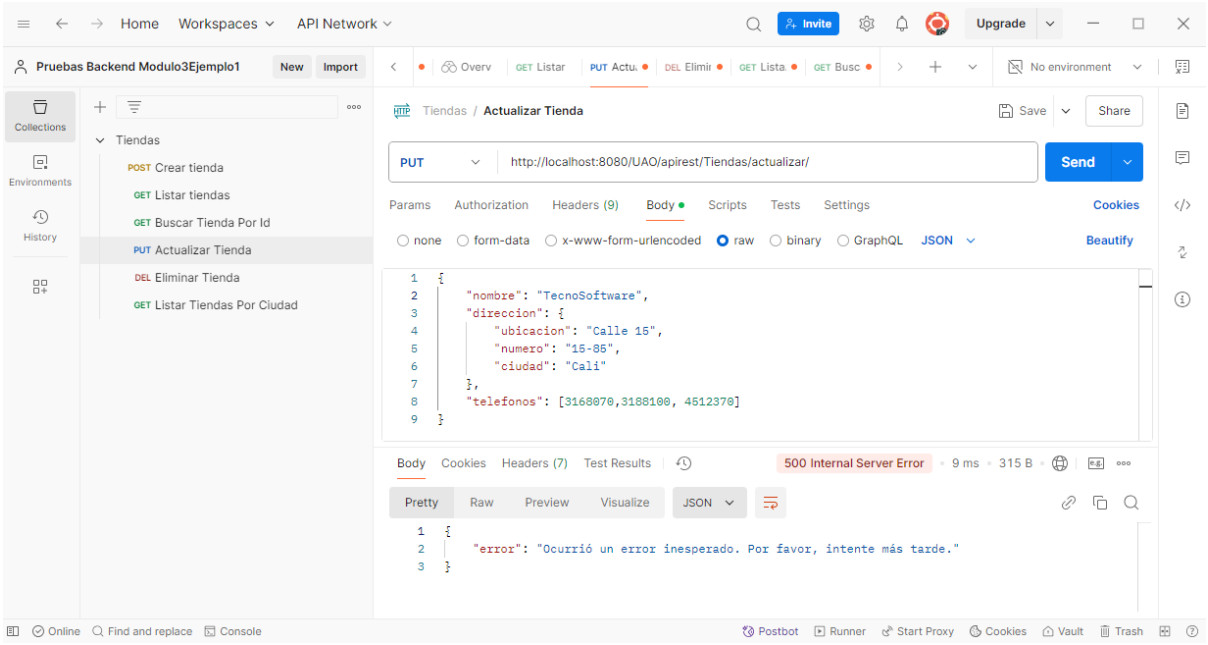

Figura 50. Actualizar Tienda por Id – Caso de Erróneo 2

- **Caso 5: Eliminar una tienda por Id**

- **Caso 5a (Éxito):** El id proporcionado existe y la tienda es eliminada correctamente.

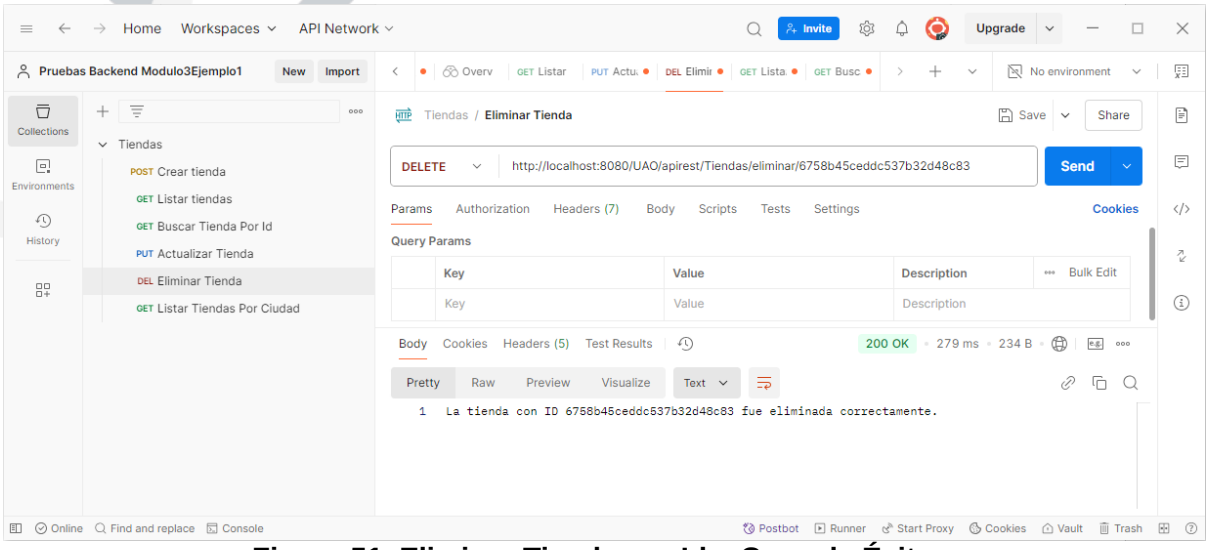

Figura 51. Eliminar Tienda por Id – Caso de Éxito

- **Caso 5b (Error 1):** El id proporcionado no existe en la base de datos. Se deberá generar el mensaje de error definido en el método “buscarTiendaPorId”.

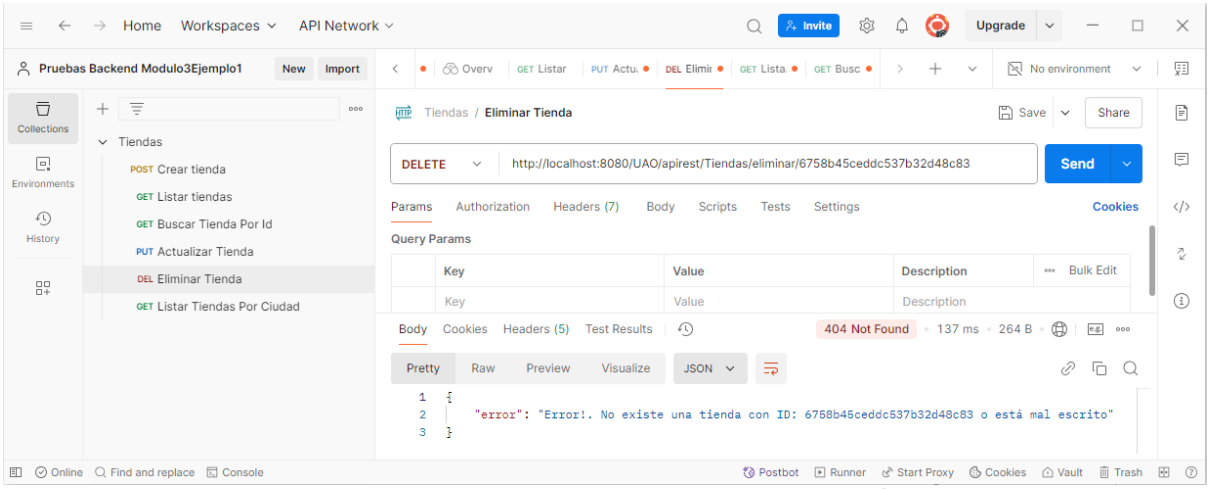

Figura 52. Eliminar Tienda por Id – Caso de Erróneo 1

- **Caso 5c (Error 2):** No se proporciona un id para realizar la eliminación. El error será manejado por el “GlobalExceptionHandler”.

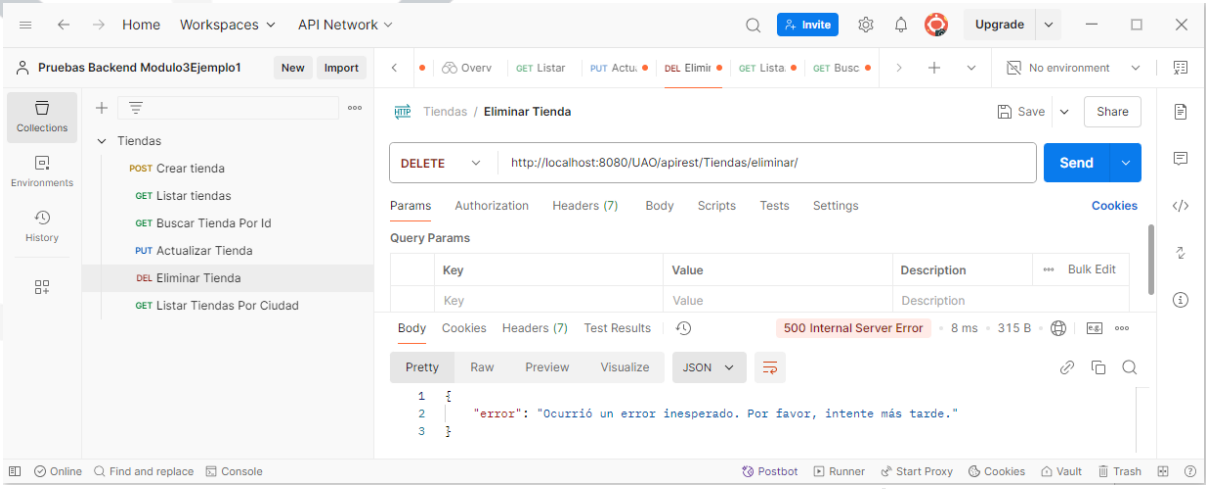

Figura 53. Eliminar Tienda por Id – Caso de Erróneo 2

- **Caso 6: Consultar tiendas por ciudad**
  Se realizará una consulta personalizada para listar todas las tiendas ubicadas en una ciudad específica, por ejemplo, Cali.

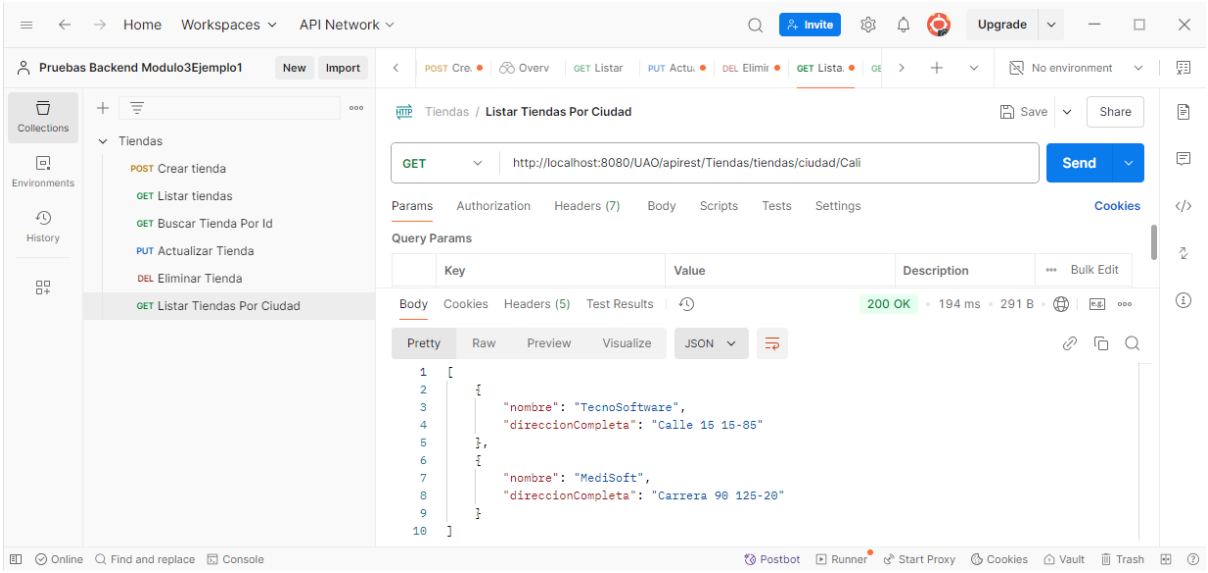

Figura 54. Consulta Tiendas Por Ciudad

Actividad Asíncrona

En el sitio del curso se encuentra disponible una nueva carpeta comprimida del proyecto, la cual incluye solo el mapeo de las colecciones “Sedes” y “Empleados”. Como parte de la actividad, deberá completarse el proyecto siguiendo lo desarrollado en esta guía e implementar las operaciones CRUD faltantes. Además, se recomienda practicar la realización de consultas personalizadas para fortalecer y profundizar el aprendizaje en la creación de un backend de una API REST.

Referencias

[1] Microsoft, “Why did we build Visual Studio Code?” [Online]. Available: https://bit.ly/3Z0FDX2
[2] Microsoft, “Programming Languages.” [Online]. Available: https://bit.ly/4180FWm
[3] Amazon, “¿Qué es API RESTful?” [Online]. Available: https://shorturl.at/lmZyd
[4] IBM, “¿Qué es una API REST?” [Online]. Available: https://shorturl.at/E3pJM